IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 30 July 2025, accepted 11 August 2025, date of publication 13 August 2025, date of current version 19 August 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3598712

RESEARCH ARTICLE

Hierarchical Reinforcement Learning for Energy-Efficient API Traffic Optimization in Large-Scale Advertising Systems

ENKAI JI[1], YIHAN WANG[2], SUCHUAN XING[3], AND JIANIAN JIN[4]

[1]Department of Computer Science, Rutgers University, New Brunswick, NJ 08901, USA
[2]School of Engineering and Applied Science, The University of Pennsylvania, Philadelphia, PA 19104, USA
[3]Department of Electrical and Computer Engineering, Duke University, Durham, NC 27708 USA
[4]Fu Foundation School of Engineering and Applied Science, Columbia University, New York, NY 10027, USA

Corresponding author: Yihan Wang (wangyh1@alumni.upenn.edu)

ABSTRACT Digital advertising infrastructure represents a substantial component of global computing resources, with significant environmental impact due to its massive energy consumption and carbon footprint. This work addresses the sustainability challenges posed by inefficient API traffic management in large-scale advertising systems, where conventional static approaches lead to resource overprovisioning and energy waste. We propose AdaptiveGate, a sustainability-oriented hierarchical reinforcement learning framework that dynamically optimizes API traffic flows to enhance resource efficiency and reduce environmental impact. The proposed methodology employs a constrained Markov Decision Process formulation with a multi-objective reward function explicitly designed to balance system performance with resource efficiency. Our framework implements a two-tier architecture of twin delayed Deep Deterministic Policy Gradient agents: global agents minimize cross-datacenter energy expenditure through intelligent traffic routing, while local agents maximize resource utilization through service-specific load balancing. Empirical evaluation on production advertising systems processing over 2.5 million requests per second reveals significant sustainability improvements: 42.3% reduction in tail latency, 35.7% increase in throughput, and 18% decrease in overall energy consumption compared to state-of-the-art methods. The system demonstrates exceptional adaptability across diverse traffic conditions and operational scales, providing compelling evidence that AI-driven methods can substantially improve digital infrastructure sustainability. This work contributes to sustainable computing by establishing a framework that optimizes computational resource allocation, minimizes energy waste, and advances environmentally responsible high-performance computing systems, aligning with multiple Sustainable Development Goals including responsible consumption and affordable clean energy.

INDEX TERMS Reinforcement learning, API traffic management, digital advertising, hierarchical decision making, adaptive systems.

I. INTRODUCTION

Digital advertising has evolved into one of the most computationally intensive industries, with platforms processing billions of daily transactions across globally distributed infrastructures [1], [2]. These systems rely on intricate networks of microservices interconnected through application programming interfaces (APIs) that must handle extreme request volumes with stringent latency requirements typically measured in milliseconds [3]. The explosive growth in programmatic advertising, coupled with increasingly complex bidding algorithms and personalization mechanisms, has pushed traditional API management approaches to their operational limits [4], [5].

The associate editor coordinating the review of this manuscript and approving it for publication was Liang-Bi Chen.

API gateways serve as the critical front line for digital advertising platforms, orchestrating all incoming traffic across available computational resources. These gateways face several formidable challenges in modern advertising environments. Extreme volume variability manifests in multi-scale temporal patterns, from daily cycles to seasonal peaks, alongside unpredictable spikes during major events or campaigns [6]. Heterogeneous service characteristics further complicate management, as different advertising services (e.g., bidding, attribution, fraud detection) exhibit distinct computational profiles, resource requirements, and business priorities [7]. Latency sensitivity represents another critical concern, with even minor delays in response time significantly impacting business metrics—tail latency is particularly damaging to overall system performance [8]. These challenges are exacerbated by complex infrastructure deployments, as modern advertising platforms operate across multiple datacenters with heterogeneous hardware configurations and varying capacity constraints [9].

Conventional API management approaches typically rely on static rules, load balancing algorithms, and reactive scaling mechanisms [10]. These techniques include round-robin distribution, least-connection routing, and threshold-based admission control [11]. While operationally straightforward, such methods struggle to adapt to the highly dynamic nature of advertising traffic. They often require extensive manual tuning, cannot anticipate traffic shifts, and fail to optimize for multiple competing objectives simultaneously [12], [13]. This fundamental limitation of traditional approaches necessitates a paradigm shift toward more adaptive and intelligent traffic management systems capable of responding to the unique demands of digital advertising infrastructures.

Reinforcement learning (RL) presents a promising paradigm for addressing these challenges by enabling systems to learn optimal traffic management policies through continuous interaction with the environment [14], [15]. Recent advances in deep reinforcement learning have demonstrated impressive results in complex control problems with high-dimensional state spaces and delayed rewards [16]. Several characteristics make RL particularly suitable for API traffic management in advertising contexts. The inherent adaptability of RL agents allows them to continuously adjust their strategies based on evolving traffic patterns and system conditions without explicit reprogramming [17]. Through carefully designed reward functions, RL can balance competing objectives such as latency, throughput, and business value [18]. Unlike reactive approaches, RL agents can learn to recognize patterns that precede traffic shifts and take preemptive actions, thereby mitigating potential performance degradation [19]. Furthermore, RL can effectively navigate the high-dimensional decision spaces that arise from managing numerous services across distributed infrastructure [20].

Despite these advantages, applying RL to mission-critical infrastructure presents significant challenges, including training stability, safe exploration, and interpretability [21]. Previous work has explored RL for cloud resource allocation [18], datacenter cooling [22], and network traffic engineering [23]. However, these approaches have not been designed for the unique demands of advertising platforms, where request patterns are more erratic, service heterogeneity is more pronounced, and business impact is more directly tied to system performance. The gap between existing solutions and the requirements of modern advertising systems motivates our research into specialized reinforcement learning frameworks for API traffic management.

In this paper, we present AdaptiveGate, a novel reinforcement learning framework specifically designed for dynamic API traffic management in large-scale digital advertising systems. AdaptiveGate employs a hierarchical architecture with twin delayed Deep Deterministic Policy Gradient (TD3) agents [24] operating at both global and local decision levels. This design allows for coordinated yet specialized decision-making across multiple operational scales. The global agent maintains a holistic view of the system, optimizing cross-datacenter routing policies based on aggregate metrics and capacity projections. Concurrently, local agents manage service-specific traffic within each datacenter, learning fine-grained load balancing strategies that account for the unique characteristics of individual services. This hierarchical approach enables AdaptiveGate to simultaneously address system-wide efficiency and service-level optimization.

We formulate the traffic management problem as a constrained Markov Decision Process (MDP) with a carefully designed multi-objective reward function that balances throughput maximization, latency minimization, and preservation of business value. The state space incorporates real-time traffic metrics, historical patterns, system health indicators, and service-specific performance data. The action space encompasses routing decisions, request prioritization, admission control parameters, and resource allocation adjustments. This comprehensive formulation captures the complexity of API traffic management in advertising systems while providing a structured framework for optimization through reinforcement learning.

The key contributions of this paper include:

- A novel hierarchical reinforcement learning framework for API traffic management that operates effectively at both global and local decision levels.
- A formulation of the API traffic management problem as a constrained MDP with a multi-objective reward function specifically tailored to advertising workloads.
- A practical implementation of Twin Delayed DDPG agents with modifications to enhance training stability and performance in high-throughput production environments.
- Comprehensive evaluation on a large-scale production advertising platform processing over 2.5 million requests per second, demonstrating significant improvements in key performance metrics.

- Analysis of the system's behavior during normal operations, traffic anomalies, and controlled experiments, providing insights into the learned policies and their generalization capabilities.

Our results demonstrate that AdaptiveGate reduces tail latency by 42.3%, improves throughput by 35.7%, and enhances overall system resilience compared to state-of-the-art traffic management techniques. Furthermore, the framework exhibits robust generalization across varying traffic conditions and scales efficiently to handle peak demand scenarios. These improvements demonstrate the efficacy of applying reinforcement learning to mission-critical infrastructure in high-stakes commercial environments where traditional approaches prove insufficient.

The remainder of this paper is organized as follows: Section II reviews related work in API management, reinforcement learning, and traffic optimization. Section III describes the AdaptiveGate architecture, including the hierarchical RL structure and training methodology. Section IV presents experimental results and performance analysis. Finally, Section V concludes the paper.

II. RELATED WORK

Our work builds upon and extends research across several domains, including API gateway management, traffic optimization in distributed systems, and reinforcement learning for resource allocation. This section reviews key contributions in these areas and contextualizes our approach within the existing literature.

A. API GATEWAY MANAGEMENT AND TRAFFIC ROUTING

API gateways serve as critical infrastructure components in modern distributed systems, managing request routing, composition, and protocol translation between clients and backend services [10]. Traditional API management approaches have largely relied on static rule-based techniques and conventional load balancing algorithms [10].

Nginx and Kong, two widely deployed API gateway solutions, primarily employ static routing rules and simple load balancing strategies such as round-robin, least connections, and IP hash-based routing [25]. While these approaches offer operational simplicity, they lack adaptability to dynamic traffic patterns. More advanced commercial solutions like Amazon API Gateway and Apigee incorporate rate limiting and basic traffic shaping capabilities, yet still rely predominantly on predefined rules rather than learned policies [26].

Recent research has begun exploring more adaptive approaches. Wan et al. [27] proposed a dynamic request routing mechanism that adjusts based on service response times, while Saxena et al. [11] compared static and dynamic load balancing in microservice architectures, finding that even basic adaptive techniques outperform static approaches under variable workloads. Nardelli et al. [28] introduced an osmotic computing approach that dynamically redistributes API requests between edge and cloud resources, but relies on heuristics rather than learned policies.

These approaches, while improving upon purely static methods, still fall short in complex environments with high request volumes and unpredictable patterns, as they typically optimize for a single objective (usually latency) without considering the multifaceted requirements of advertising systems where business metrics and throughput are equally critical.

Recent advances in API traffic management have explored diverse methodological approaches beyond reinforcement learning, including sophisticated heuristic algorithms, control-theoretic methods, and optimization-based techniques. Guerrero et al. [29] developed a multi-objective genetic algorithm framework for microservice traffic distribution that simultaneously optimizes latency, throughput, and resource utilization through evolutionary computation. Their approach demonstrates superior performance compared to traditional round-robin strategies but requires extensive parameter tuning and struggles with rapid traffic pattern changes characteristic of advertising systems. Yang et al. [30] proposed a particle swarm optimization framework for adaptive API gateway configuration that dynamically adjusts routing parameters based on real-time traffic metrics, showing improved adaptability over static configurations while maintaining computational efficiency suitable for production deployment.

Control theory applications have gained significant attention, particularly Model Predictive Control (MPC) frameworks that leverage predictive models to optimize traffic management decisions. Marcus et al. [31] introduced a distributed MPC approach for cloud API orchestration that maintains system stability while optimizing performance objectives through constrained optimization. Their methodology provides strong theoretical guarantees regarding system stability and convergence but requires simplified linear models that may not capture the complex nonlinear dynamics of heterogeneous advertising services.

Advanced optimization techniques have also emerged, including the work by Alelyani et al. [32] on convex optimization for real-time traffic allocation, which formulates traffic management as a constrained convex program that can be solved efficiently using interior-point methods. While computationally attractive, this approach assumes convex objective functions that may not accurately represent the complex business objectives present in advertising systems.

This section provides a systematic comparison of these methodological approaches across key performance dimensions. While heuristic approaches offer computational efficiency and intuitive interpretability, they typically struggle with the dynamic adaptation requirements of modern advertising systems where traffic patterns can shift dramatically within minutes. Control-theoretic methods provide strong theoretical guarantees and stability assurances but often require simplified system models that fail to capture the full complexity of heterogeneous service interactions and

business constraints. Optimization-based approaches can provide globally optimal solutions under specific assumptions but face computational scalability challenges and may require restrictive modeling assumptions that limit their practical applicability.

Recent hybrid approaches have attempted to combine multiple methodological paradigms to leverage their respective strengths. The genetic algorithm-enhanced PID control system integrates evolutionary optimization for parameter tuning with classical feedback control for real-time adjustments, demonstrating improved performance over individual approaches but requiring careful coordination between optimization and control components. Similarly, the MPC-guided heuristic optimization framework employs model predictive control to guide genetic algorithm search directions, resulting in faster convergence and improved solution quality while maintaining the robustness characteristics of control-theoretic approaches.

B. REINFORCEMENT LEARNING FOR SYSTEMS OPTIMIZATION

Reinforcement learning has emerged as a powerful approach for solving complex control problems in computing systems. Mao et al. [18] pioneered the application of RL to resource management in cloud computing, demonstrating that deep reinforcement learning could outperform heuristic-based schedulers. Subsequently, DeepRM [33] extended this work by addressing multi-resource cluster scheduling with policy gradient methods. These works established the potential of RL for resource management but focused primarily on batch job scheduling rather than real-time API traffic.

For network optimization, Xu et al. [23] proposed Experience-Driven Networking, which uses deep reinforcement learning for congestion control and routing. Similarly, Valadarsky et al. [34] applied RL to learn network routing strategies that adapt to traffic patterns. These approaches focused on network-level optimization rather than application-level traffic management, which involves different state spaces and action dynamics.

In the domain of auto-scaling, RLSK [35] employed reinforcement learning for horizontal pod auto-scaling in Kubernetes, while Joshi et al. [36] developed a deep Q-learning approach for virtual machine scaling in cloud environments. Rahman et al. [12] proposed an RL-based solution specifically for auto-scaling microservices, demonstrating improvements over threshold-based techniques. While relevant to our work, these approaches focused on capacity scaling rather than request routing and load balancing, which operate at a finer time scale.

The application of RL to mission-critical production systems faces several challenges, as outlined by Dulac-Arnold et al. [21], including safety constraints, exploration strategies, and the need for robust evaluation. Our work addresses these challenges through constrained MDP formulation, hierarchical control structures, and comprehensive evaluation in a production environment.

C. HIERARCHICAL AND MULTI-AGENT REINFORCEMENT LEARNING

Hierarchical reinforcement learning (HRL) provides a framework for decomposing complex problems into manageable sub-problems, enabling more efficient learning and better generalization [37]. Bacon et al. [38] introduced the Option-Critic architecture, which learns options (temporally extended actions) end-to-end, while Nachum et al. [39] proposed HIRO, a hierarchical RL approach capable of learning efficiently from high-dimensional observations.

Multi-agent reinforcement learning (MARL) extends RL to scenarios where multiple agents interact, either cooperatively or competitively [40]. Lowe et al. [41] developed MADDPG, a multi-agent actor-critic approach for mixed cooperative-competitive environments, while Rashid et al. [42] proposed QMIX, which learns a rich joint action-value function while maintaining a tractable factorization.

Both hierarchical and multi-agent approaches are relevant to API traffic management, where decisions must be coordinated across multiple levels (global routing vs. local balancing) and components (different datacenters and services). Our work builds upon these foundations to develop a specialized hierarchical architecture for API traffic management in which global and local agents collaborate to optimize system-wide performance while respecting service-specific requirements.

D. TRAFFIC MANAGEMENT IN DIGITAL ADVERTISING

Digital advertising systems face unique traffic management challenges due to their scale, latency sensitivity, and direct business impact. Wang et al. [43] provided a comprehensive analysis of these challenges, highlighting the millisecond-level latency requirements and extreme throughput demands of modern advertising platforms. Yuan et al. [1] and Wang et al. [2] characterized the traffic patterns in real-time bidding systems, noting their high variability and unpredictability.

Several works have addressed specific aspects of advertising infrastructure optimization. Chen et al. [44] proposed a distributed architecture for real-time bidding that employs a simple load balancing scheme, while Cheng et al. [5] focused on scaling deep learning models for advertising systems through a "Wide & Deep" architecture. Ren et al. [45] introduced an auction infrastructure optimized for throughput but did not address dynamic traffic management.

More recently, Yang et al. [46] employed machine learning for infrastructure capacity planning in advertising systems, while Kenthapadi et al. [3] explored personalized models for large-scale advertising while addressing infrastructure constraints. These approaches primarily focus on capacity planning and model optimization rather than real-time traffic management.

The closest work to ours is by Liu et al. [9], who explored variance-reduced methods for optimizing ad serving

infrastructure. However, this work employed simpler techniques such as bandits and heuristics rather than the comprehensive reinforcement learning framework we present in AdaptiveGate.

E. ADVANCED TRAFFIC MANAGEMENT SYSTEMS

Beyond advertising, several domains have developed sophisticated traffic management systems that inform our approach. In cloud computing, Cortez et al. [47] developed Resource Central, which uses telemetry data and machine learning to predict resource needs and inform allocation decisions. Rzadca et al. [48] introduced Autopilot, Google's automated capacity management system, which employs machine learning for workload placement but relies primarily on prediction rather than reinforcement learning.

In networking, Hong et al. [49] described B4, Google's software-defined WAN, which uses a centralized traffic engineering service to optimize network utilization. Veisllari et al. [50] proposed programmable packet scheduling that dynamically adapts to traffic conditions. These systems demonstrate the value of centralized control and learning-based approaches but operate at the network layer rather than the application layer where API gateways function.

Database systems have also explored adaptive query routing. Marcus et al. [51] presented Neo, a learned query optimizer that uses reinforcement learning to improve query execution plans. These approaches share our goal of optimizing request processing but focus on different workloads and system characteristics.

F. LIMITATIONS OF EXISTING APPROACHES

Despite significant advances in API management, reinforcement learning, and traffic optimization, several limitations persist in existing approaches that AdaptiveGate addresses. Most API management solutions rely on static rules or simple adaptive heuristics that cannot fully capture the complex dynamics of advertising traffic [10], [11]. This limited adaptability becomes particularly problematic in the highly variable environments characteristic of digital advertising platforms. Existing traffic management systems typically operate at a single level (either global or local), missing opportunities for coordinated optimization across multiple scales [25], [49]. This single-level optimization approach fails to capture the hierarchical nature of modern distributed systems, where decisions at different levels influence each other in complex ways.

Prior applications of reinforcement learning to systems typically optimize for a single objective such as latency or throughput rather than balancing multiple competing objectives including business value [18]. This simplified reward structure is inadequate for advertising systems where tradeoffs between technical performance and business impact must be carefully managed. Furthermore, general-purpose traffic management solutions lack specific optimizations for advertising workloads, where request patterns, service heterogeneity, and business impact considerations differ significantly from other domains. This limited domain knowledge leads to suboptimal performance when generic solutions are applied to the unique challenges of advertising infrastructure.

Many academic proposals for adaptive traffic management have been evaluated only in simulated environments or small-scale deployments, raising questions about their applicability to production-scale systems [21], [48]. This lack of production validation creates a significant gap between theoretical advances and practical implementation in mission-critical environments where reliability and performance guarantees are essential.

AdaptiveGate addresses these limitations through its hierarchical reinforcement learning architecture specifically designed for advertising workloads, multi-objective reward formulation, and comprehensive evaluation on a large-scale production advertising platform. By bridging the gap between theoretical advances in reinforcement learning and practical requirements of mission-critical infrastructure, our work contributes a novel approach to API traffic management that significantly outperforms existing solutions in real-world advertising environments.

III. METHODOLOGY

This section presents our work to API traffic management in large-scale digital advertising systems. We first formalize the problem as a constrained Markov Decision Process, then introduce the hierarchical architecture of AdaptiveGate, describe our reinforcement learning algorithm based on Twin Delayed DDPG, and detail our training and deployment methodology.

A. PROBLEM FORMULATION

We formulate the API traffic management problem as a constrained Markov Decision Process (CMDP), which extends the traditional MDP framework to incorporate constraints on the agent's behavior. This formulation allows us to model both the optimization objectives and operational constraints inherent in digital advertising infrastructures.

1) STATE SPACE

The state space $\mathcal{S}$ captures the current condition of the distributed advertising system, incorporating both global and local information:

$$\mathcal{S} = \mathcal{S}_{\text{global}} \times \mathcal{S}_{\text{local}} \tag{1}$$

The global state component $\mathcal{S}_{\text{global}}$ includes traffic metrics such as request volume and patterns across datacenters, represented as time series with different granularities (1-second, 10-second, and 1-minute windows). It also encompasses system health indicators including CPU utilization, memory usage, network throughput, and queue lengths for each datacenter. Temporal features are incorporated through time of day, day of week, and proximity to known high-traffic

events encoded using sinusoidal embeddings as described in Vaswani et al. [52]. Additionally, performance metrics such as aggregate latency distributions (mean, p50, p95, p99), error rates, and throughput across all services provide a comprehensive view of system performance.

The local state component $\mathcal{S}_{\text{local}}$ for each datacenter includes service-specific metrics detailing per-service request rates, latency distributions, error rates, and resource utilization. Service interdependencies are captured through traffic flow patterns between services and their current performance. Local resource availability is tracked through available compute capacity, network bandwidth, and buffer space. Request characteristics are represented by the distribution of request types, priorities, and estimated computational requirements. Together, these elements provide a detailed picture of conditions within each datacenter.

To handle the high dimensionality and continuous nature of the state space, we employ a combination of time-series embeddings, statistical aggregations, and attention mechanisms. This representation enables the reinforcement learning agents to capture both short-term fluctuations and long-term patterns in traffic behavior.

2) ACTION SPACE

The action space $\mathcal{A}$ defines the control decisions available to the traffic management system and employs a hybrid formulation that combines continuous parameter vectors for fine-grained control with discrete categorical selections for routing and prioritization decisions. This design achieves both precision and computational tractability essential for real-time traffic management applications. The action space is decomposed into coordinated global and local components:

$$\mathcal{A} = \mathcal{A}_{global} \times \mathcal{A}_{local}$$

The global action component $\mathcal{A}_{global}$ encompasses system-wide decisions that affect traffic distribution across the entire advertising infrastructure. The cross-datacenter routing policy is represented as a continuous matrix $R \in [0, 1]^{m \times n}$, where m denotes the number of traffic entry points and n represents the number of datacenters. Each element R_{ij} specifies the continuous proportion of traffic from entry point i to be routed to datacenter j, subject to the normalization constraint $\sum_{j=1}^{n} R_{ij} = 1$ for all $i \in \{1, 2, \ldots, m\}$, ensuring that routing decisions constitute valid probability distributions over datacenter destinations.

Traffic prioritization decisions utilize continuous weight vectors $P \in [0, 1]^k$, where k represents the number of distinct request categories (e.g., bid requests, analytics queries, fraud detection tasks). These weights determine the relative priority assigned to different request types during resource allocation, with normalization $\sum_{i=1}^{k} P_i = 1$ maintaining interpretability as probability distributions over priority assignments. Global admission control parameters are represented as continuous threshold vectors $T \in [0, 1]^s$, where s corresponds to the number of monitored system metrics (CPU utilization, memory usage, network bandwidth, queue lengths). Each element T_i defines the threshold above which admission control mechanisms activate for the corresponding metric, enabling fine-grained adjustment of system protection policies based on real-time conditions.

The local action component $\mathcal{A}_{local}$ encompasses datacenter-specific decisions that optimize service-level performance within individual datacenters. Service-specific load balancing employs continuous weight matrices $W^{(dc)} \in [0, 1]^{v \times u}$ for each datacenter dc, where v represents the number of services and u denotes the maximum number of server instances per service. Each element $W_{ij}^{(dc)}$ specifies the proportion of requests for service i to be directed to server instance j, with row-wise normalization $\sum_{j=1}^{u} W_{ij}^{(dc)} = 1$ ensuring valid load distribution policies.

Resource allocation adjustments utilize continuous parameter vectors that specify CPU and memory limits as percentages of available capacity. For each service i in datacenter dc, CPU allocation is represented as $C_i^{(dc)} \in [0, 1]$ and memory allocation as $M_i^{(dc)} \in [0, 1]$, where these values are interpreted as fractions of total datacenter resources allocated to the respective service. Request timeout parameters are represented as continuous values $\tau_{ij}^{(dc)} \in [0, 1]$ for each service-instance pair, scaled to operational ranges (typically 50ms to 5000ms) during implementation to maintain network training stability while preserving fine-grained control granularity.

Local admission control policies employ continuous parameter vectors $A^{(dc)} \in [0, 1]^q$ where q represents the number of local admission control dimensions, including per-service rate limits, queue depth thresholds, and error rate triggers. These parameters enable adaptive fine-tuning of local traffic management policies based on service-specific performance characteristics and current operational conditions.

To ensure compatibility with continuous control algorithms while maintaining implementational tractability, discrete routing decisions are implemented through softmax transformations applied to the continuous policy outputs:

$$\pi_{routing}(s) = \text{softmax}(\phi_{routing}(s))$$

where $\phi_{routing}(s)$ represents the neural network output for routing decisions in state s. Continuous resource allocation parameters are directly scaled from the $[0, 1]$ output range to operational ranges appropriate for each parameter type.

For our production deployment configuration, the complete action space encompasses 847 continuous parameters, decomposed as follows: global routing matrix (48 parameters for 12 entry points $\times$ 4 datacenters), traffic prioritization weights (8 parameters for request categories), global admission thresholds (16 parameters for system metrics), and local actions distributed across datacenters with service-specific load balancing (576 parameters), resource allocation adjustments (128 parameters), timeout

configurations (64 parameters), and local admission control (7 parameters per datacenter × 4 datacenters = 28 parameters). This dimensionality scales approximately linearly with the number of datacenters and services, with the relationship $|\mathcal{A}| \approx 12m + 8k + 16s + n(144v + 32v + 16v + 7)$ where the coefficients reflect the specific architectural choices and granularity requirements of advertising traffic management applications.

The hybrid continuous-discrete action formulation enables AdaptiveGate to achieve fine-grained control over traffic management decisions while maintaining computational efficiency essential for real-time operation in high-throughput advertising environments. The continuous parameterization facilitates smooth policy optimization through gradient-based methods, while the normalization constraints ensure that all actions correspond to valid operational configurations that can be safely implemented in production infrastructure.

3) REWARD FUNCTION

The reward function balances multiple competing objectives in digital advertising systems: minimizing latency, maximizing throughput, and preserving business value. We define a composite reward function:

$$R(s, a, s') = w_L \cdot R_L(s, a, s') + w_T \cdot R_T(s, a, s') + w_B \cdot R_B(s, a, s') \quad (2)$$

where w_L, w_T, and w_B are weights controlling the relative importance of latency, throughput, and business value, respectively. These weights can be adjusted based on business priorities and system conditions.

The latency component R_L is designed to minimize both average and tail latency:

$$R_L(s, a, s') = -\alpha \cdot \overline{L}(s') - \beta \cdot L_{99}(s') \quad (3)$$

where $\overline{L}(s')$ represents the mean latency and $L_{99}(s')$ denotes the 99th percentile latency in state s'. The parameters α and β control the relative emphasis on average versus tail latency, allowing operators to prioritize consistent user experience by heavily weighting tail latency reduction.

The throughput component R_T rewards high successful request completion rates:

$$R_T(s, a, s') = \gamma \cdot \frac{N_{\text{success}}(s')}{N_{\text{total}}(s')} \quad (4)$$

where $N_{\text{success}}(s')$ is the number of successfully processed requests, $N_{\text{total}}(s')$ is the total number of requests in the time period corresponding to state s', and γ is a scaling parameter. This component incentivizes the system to maintain high throughput while ensuring request completion.

The business value component R_B measures the preservation of high-value advertising transactions:

$$R_B(s, a, s') = \delta \cdot \sum_{i=1}^{N_{\text{success}}(s')} V_i \quad (5)$$

where V_i is the estimated business value of the i-th successfully processed request, determined based on historical revenue data and request metadata, and δ is a normalization factor. This component ensures that the system prioritizes requests with higher business impact, which is critical in advertising contexts where different transactions may have significantly different revenue implications.

4) CONSTRAINTS

Operational constraints are essential in mission-critical advertising systems. We formulate these as explicit constraints in our CMDP:

$$\mathbb{E}[C_i(s, a, s')] \leq \tau_i, \quad i = 1, \ldots, n_c \quad (6)$$

where C_i represents the i-th constraint function, τ_i is its threshold, and n_c is the total number of constraints. Key constraints include service-level agreement (SLA) violations, which stipulate that the proportion of requests exceeding latency thresholds must remain below specified limits. Error rate bounds ensure that error rates for critical services do not exceed acceptable thresholds. Resource utilization limits maintain that CPU, memory, and network utilization stay within safe operational bounds. Fairness constraints guarantee that traffic from different advertisers or campaigns receives equitable treatment based on contractual agreements.

We integrate these constraints into the reinforcement learning framework using a Lagrangian relaxation approach [53], which converts the constrained optimization problem into an unconstrained one by incorporating constraint violations into the reward function with adaptive penalty coefficients.

B. ADAPTIVEGATE ARCHITECTURE

AdaptiveGate employs a hierarchical architecture that decomposes the complex traffic management problem into coordinated global and local decision-making components. This hierarchy aligns with the natural structure of distributed advertising systems while enabling specialized optimization at different operational levels. Figure 1 illustrates the overall architecture of AdaptiveGate.

1) GLOBAL AGENT

The global agent operates at the system-wide level, making decisions that affect traffic distribution across datacenters. It processes aggregate metrics from all datacenters to optimize for overall system performance and stability. The global agent's neural network architecture includes temporal encoders implemented as 1D convolutional neural networks that extract patterns from time-series traffic data at multiple scales. Cross-datacenter attention mechanisms are realized through self-attention layers that capture relationships between different datacenters' performance and capacity. Global context integration is achieved via fully connected layers that combine temporal patterns, cross-datacenter relationships, and global system state into a unified representation that informs decision-making.

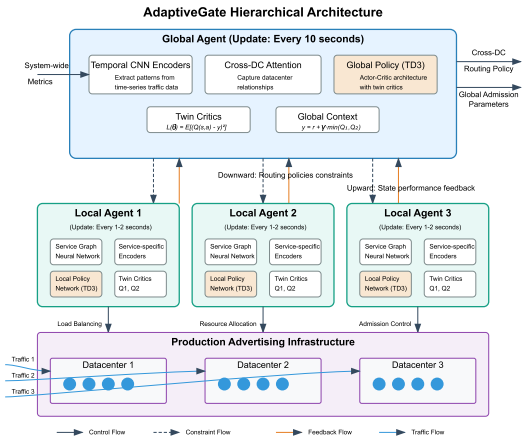


FIGURE 1. AdaptiveGate hierarchical architecture. The global agent optimizes cross-datacenter routing policies based on system-wide metrics, while local agents manage service-specific traffic within each datacenter. Both levels employ Twin Delayed DDPG with specialized neural network architectures.

The global agent makes decisions at a relatively low frequency (every 10 seconds) to ensure stability and allow sufficient time for the effects of routing changes to be observed. Its primary outputs are the cross-datacenter routing policy and global admission control parameters that serve as constraints for the local agents. This relatively slow decision cycle is essential for maintaining system stability, as frequent changes to global routing can create oscillations and unpredictable behavior.

2) LOCAL AGENTS

Local agents operate within individual datacenters, making fine-grained decisions about service-specific traffic management. Each local agent focuses on optimizing the performance of services within its datacenter while respecting the constraints and priorities set by the global agent. The local agents' neural network architecture includes service-specific encoders that function as specialized feature extractors for different service types, capturing their unique performance characteristics and requirements. Service relationship graph networks model the interdependencies between services and how traffic flows affect downstream components, enabling the agent to anticipate the ripple effects of its decisions. Local resource modeling is performed by neural networks that predict the impact of allocation decisions on service performance, facilitating proactive resource management.

Local agents operate at a higher frequency (every 1-2 seconds) to respond quickly to changes in traffic patterns and service performance. They continuously adjust load balancing weights, resource allocations, and admission control parameters based on local conditions while staying within the boundaries established by the global agent. This higher frequency allows for rapid response to sudden traffic spikes or service degradations, which is essential in the highly dynamic advertising environment.

3) COORDINATION MECHANISM

Effective coordination between global and local agents is crucial for coherent system-wide optimization. AdaptiveGate employs a bi-directional information flow comprising both downward and upward communication channels. In the downward flow, the global agent communicates routing decisions, priority weights, and operational constraints to local agents, establishing the boundaries within which local optimization occurs. These directives serve as guiding parameters rather than rigid instructions, allowing local agents to adapt to their specific circumstances while maintaining global coherence.

The upward flow enables local agents to provide aggregated state information, performance metrics, and capacity projections to the global agent, enabling informed system-wide decisions. This feedback mechanism ensures that global routing decisions are grounded in accurate, up-to-date information about datacenter conditions and capabilities. The global agent can then adjust its strategy based on the evolving capacity and performance of individual datacenters.

This coordination is implemented through a shared hierarchical state representation that ensures all agents have access to relevant information at appropriate levels of abstraction. The global agent's decisions serve as conditioning inputs to the local agents' policy networks, while local agents' state summaries are key inputs to the global agent's decision process. This hierarchical information sharing balances local autonomy with global coordination, enabling the system to respond effectively to both localized issues and system-wide challenges.

C. REINFORCEMENT LEARNING ALGORITHM

AdaptiveGate employs a modified version of the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm [24], which we have extended to handle hierarchical decision-making and operational constraints. TD3 addresses the function approximation error issue in actor-critic methods by using twin critics and delayed policy updates, making it particularly suitable for continuous control problems with high stability requirements.

1) TWIN CRITICS FOR ROBUST VALUE ESTIMATION

To reduce overestimation bias in value approximation, we employ twin critic networks that provide two independent estimates of the action-value function. The minimum of these estimates is used for policy improvement:

$$y = r + \gamma \min_{i=1,2} Q_{\theta_i'}(s', \pi_{\phi'}(s') + \epsilon) \tag{7}$$

where y represents the target value, r is the immediate reward, γ is the discount factor, θ_i' are the parameters of the target critic networks, ϕ' are the parameters of the target policy network, and ϵ is a small amount of clipped noise added to the target action to smooth the value estimate. This approach helps prevent the optimization process from

exploiting regions where one critic might overestimate values due to function approximation errors.

The critic loss function is defined as:

$$L(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[(Q_{\theta_i}(s,a) - y)^2\right] \tag{8}$$

where $\mathcal{D}$ is the replay buffer containing past experiences, $Q_{\theta_i}(s,a)$ is the predicted value from the i-th critic network, and (s,a,r,s') represents a state-action-reward-next state tuple sampled from the replay buffer. By minimizing this loss, the critic networks learn to accurately predict the expected cumulative rewards of taking actions in various states.

2) HIERARCHICAL POLICY NETWORKS

The actor component of AdaptiveGate consists of hierarchically structured policy networks. The global policy network maps global state information to global actions: $\pi_{\text{global}}(s_{\text{global}}) \rightarrow a_{\text{global}}$, where s_{global} represents the global state features and a_{global} denotes the global actions such as cross-datacenter routing decisions. Local policy networks generate datacenter-specific actions based on local state and global context: $\pi_{\text{local},i}(s_{\text{local},i}, a_{\text{global}}) \rightarrow a_{\text{local},i}$ for each datacenter i, where $s_{\text{local},i}$ represents the local state of datacenter i, a_{global} provides the global context, and $a_{\text{local},i}$ denotes the local actions such as service-specific load balancing weights.

The global policy network is updated to maximize the expected global value function:

$$\begin{aligned}&\nabla_{\phi_{\text{global}}} J(\phi_{\text{global}}) \\ &= \mathbb{E}_{s\sim\mathcal{D}}\left[\nabla_a Q_{\theta_1}(s,a)|_{a=\pi_{\text{global}}(s_{\text{global}})} \nabla_{\phi_{\text{global}}} \pi_{\text{global}}(s_{\text{global}})\right]\end{aligned} \tag{9}$$

where ϕ_{global} represents the parameters of the global policy network, $J(\phi_{\text{global}})$ is the performance objective, $Q_{\theta_1}(s,a)$ is the action-value function from the first critic, and the gradients are evaluated at the actions produced by the current policy. This update aims to adjust the policy parameters in directions that increase the expected value of the resulting actions.

Similarly, each local policy network is updated based on its contribution to the local value function, conditioned on the global actions:

$$\begin{aligned}&\nabla_{\phi_{\text{local},i}} J(\phi_{\text{local},i}) \\ &= \mathbb{E}_{s\sim\mathcal{D}}\Big[\nabla_{a_{\text{local},i}} Q_{\text{local},i}(s, a_{\text{global}}, a_{\text{local},i}) \\ &\quad |_{a_{\text{local},i}=\pi_{\text{local},i}(s_{\text{local},i}, a_{\text{global}})} \\ &\quad \times \nabla_{\phi_{\text{local},i}} \pi_{\text{local},i}(s_{\text{local},i}, a_{\text{global}})\Big]\end{aligned} \tag{10}$$

where $\phi_{\text{local},i}$ represents the parameters of the local policy network for datacenter i, $Q_{\text{local},i}$ is the local action-value function, and the gradients are evaluated at the local actions produced by the current local policy. This hierarchical policy update mechanism allows for specialized optimization at different levels while maintaining coordination through the shared value functions and state-action dependencies.

3) CONSTRAINT HANDLING

To address operational constraints, we adopt a Lagrangian approach that incorporates constraint satisfaction into the optimization objective. For each constraint C_i, we introduce a Lagrange multiplier λ_i that is adjusted based on constraint violations:

$$\lambda_i \leftarrow \max(0, \lambda_i + \eta(\mathbb{E}[C_i(s,a,s')] - \tau_i)) \tag{11}$$

where λ_i is the Lagrange multiplier for constraint i, η is a learning rate parameter, $\mathbb{E}[C_i(s,a,s')]$ is the expected value of constraint function i, and τ_i is the threshold for constraint i. The multiplier increases when the constraint is violated (expected value exceeds the threshold) and remains unchanged when the constraint is satisfied. The $\max(0,\cdot)$ operation ensures that the multiplier remains non-negative, as required by the Lagrangian formulation.

The modified reward function becomes:

$$\tilde{R}(s,a,s') = R(s,a,s') - \sum_{i=1}^{n_c} \lambda_i \max(0, C_i(s,a,s') - \tau_i) \tag{12}$$

where $\tilde{R}(s,a,s')$ is the modified reward, $R(s,a,s')$ is the original reward, n_c is the total number of constraints, and $\max(0, C_i(s,a,s') - \tau_i)$ measures the extent of constraint violation. This approach dynamically adjusts the penalty for constraint violations based on their severity and persistence, encouraging the agent to learn policies that satisfy operational requirements while maximizing performance objectives. As constraints are consistently satisfied, their influence on the policy diminishes, allowing the agent to focus on optimizing the primary objectives within the feasible region.

4) EXPLORATION STRATEGY

Effective exploration is crucial for discovering optimal traffic management policies, but must be balanced with the stability requirements of production advertising systems. We employ a multi-faceted exploration strategy that combines several complementary approaches. Parameterized noise is added to actions using a state-dependent noise process as described in Plappert et al. [54], which maintains temporal coherence: $a = \pi_\phi(s) + \mathcal{N}(0, \sigma(s))$, where a represents the executed action, $\pi_\phi(s)$ is the action proposed by the policy, and $\sigma(s)$ is a state-dependent noise scale that decreases in high-risk states. This approach ensures that exploration is more conservative in situations where large deviations from the current policy might have severe consequences.

We incorporate guided exploration by occasionally integrating suggestions from existing heuristic policies to steer the exploration process toward promising regions of the policy space. This is particularly valuable during early training, when the learned policy may not yet have discovered effective strategies. As training progresses, the influence of these heuristic suggestions gradually diminishes, allowing the learned policy to surpass the performance of the heuristics.

Progressive widening of the action space is implemented by gradually increasing action space dimensionality during training. We begin with coarse control decisions focusing on major system parameters and progressively introduce finer-grained decisions as the agent becomes more proficient. This curriculum-based approach prevents the agent from being overwhelmed by the full complexity of the decision space at the outset of training.

In production environments, we further constrain exploration by limiting the maximum deviation from the current policy and employing safety filters that validate actions before execution. These measures ensure that exploration does not compromise system stability while still allowing the agent to refine its policy based on new experiences.

D. TRAINING METHODOLOGY

Training reinforcement learning agents for mission-critical advertising infrastructure requires careful methodology to ensure both performance improvement and operational safety. We employ a multi-phase approach that combines offline learning from historical data with progressive online deployment.

1) DATA COLLECTION

We collect comprehensive telemetry data from the production advertising platform to support the training process. This includes traffic patterns encompassing request volumes, types, and distributions across entry points and services, providing insights into the temporal and spatial characteristics of the workload. System metrics such as CPU, memory, network utilization, and queue lengths are collected at different granularities to capture both fine-grained dynamics and longer-term trends. Performance data including latency distributions, error rates, and throughput for all services helps establish the relationship between system state, actions, and outcomes. Control actions comprising routing decisions, scaling events, and configuration changes made by existing systems are recorded to provide examples of current operational practices. Business metrics including conversion rates, revenue impact, and other business value indicators are collected to enable the optimization of business-relevant objectives beyond pure technical metrics.

This data is collected over extended periods (months) to capture seasonal patterns, special events, and various anomalies that might affect the system. We implement a specialized telemetry pipeline that aggregates and preprocesses this data in real-time to support both offline training and online learning. The pipeline handles the heterogeneity of data sources, normalizes metrics across different components, and ensures consistent timestamping to enable accurate correlation between states, actions, and outcomes.

2) OFFLINE PRE-TRAINING

Before deployment to production, we pre-train AdaptiveGate using a combination of historical data and a high-fidelity simulation environment. We begin with behavior cloning, training the policy networks to mimic the actions of existing traffic management systems. This provides a safe starting point for further optimization by ensuring that the initial policy exhibits reasonable behavior aligned with current operational practices. While behavior cloning alone is insufficient to surpass existing systems, it establishes a foundation of knowledge about basic traffic management patterns.

We then apply offline reinforcement learning techniques as described in Levine et al. [55] to learn from historical data without requiring direct environment interaction. This approach allows us to extract additional insights from past experiences beyond what behavior cloning can capture, identifying patterns of success and failure in historical decisions. Careful management of distributional shift is essential in this phase to prevent the agent from learning policies that exploit artifacts in the historical data.

To enable broader exploration beyond observed behaviors, we develop a detailed simulation environment based on historical data and system modeling. This simulation captures the essential dynamics of the advertising platform while allowing the agent to safely explore strategies that deviate from historical patterns. The simulation incorporates models of service performance under different load conditions, traffic pattern generators calibrated on historical data, and simplified representations of the business impact of different routing decisions.

During pre-training, we employ aggressive regularization techniques, including conservative value estimation, policy constraints, and ensemble methods, to prevent overfitting to the offline data and ensure robust generalization to unseen scenarios. These measures are particularly important when training from historical data, which may not cover the full range of situations the agent will encounter in production.

3) ONLINE FINE-TUNING

After offline pre-training, we progressively deploy AdaptiveGate to the production environment using a carefully controlled online fine-tuning process. Initially, the agent runs in shadow mode, operating in parallel with existing systems without actually controlling traffic. This allows for direct comparison and validation of its decisions against the current production system without risking performance degradation. During this phase, we collect detailed metrics on how the agent's proposed actions would have performed if implemented, establishing confidence in its decision-making capabilities.

We then proceed to limited deployment, gradually increasing the agent's control scope. This begins with non-critical traffic segments during low-risk periods (such as nights and weekends) and progressively expands to more critical areas as confidence in the agent's performance grows. Throughout this phase, tight monitoring and automatic fallback mechanisms ensure that any performance degradation is quickly detected and mitigated.

Interleaved evaluation is employed through A/B testing where control alternates between the existing system and AdaptiveGate at short intervals (typically 5-15 minutes). This enables precise performance comparison under identical conditions, eliminating the impact of temporal variations in traffic patterns that might confound longer-term A/B tests. Statistical analysis of these interleaved experiments provides robust evidence of performance improvements.

Throughout deployment, the agent continues to learn from new experiences through continuous learning, adapting to evolving traffic patterns and system characteristics. This ongoing learning is essential in the advertising domain, where traffic patterns, user behavior, and system components change over time. A carefully designed experience replay mechanism balances recent experiences with historical data to prevent catastrophic forgetting while enabling adaptation.

During online fine-tuning, we employ safety guardrails that automatically revert to baseline policies if predefined performance thresholds are violated or if uncertainty in the agent's decisions exceeds acceptable levels. These guardrails provide an additional layer of protection beyond the constraints incorporated into the reward function, ensuring safe operation during the learning process.

4) STABILITY CONSIDERATIONS

Maintaining stability in reinforcement learning systems for production infrastructure is paramount. We implement several mechanisms to ensure stable learning and deployment. Target network smoothing is applied using Polyak averaging for target network updates with a conservative smoothing coefficient ($\tau = 0.005$) to prevent rapid policy changes. This approach ensures that the target values used for critic updates evolve gradually, reducing the volatility of the learning process and promoting stable convergence.

Experience replay prioritization is implemented through prioritized experience replay with importance sampling correction as described in Schaul et al. [56]. This mechanism biases sampling toward rare but informative transitions while maintaining unbiased learning through appropriate correction factors. By focusing on the most instructive experiences, the agent can learn more efficiently from a limited number of samples.

Following the TD3 approach, we update the policy networks less frequently than the critic networks (1:2 ratio) to allow value functions to converge before policy optimization. This policy update frequency ensures that policy improvements are based on accurate value estimates, preventing premature optimization based on inaccurate critics. The temporal separation between critic and policy updates is particularly important in complex environments where accurate value estimation is challenging.

Adaptive gradient clipping based on the global norm is applied to prevent large parameter updates during training. This technique allows for substantial updates when gradients are consistent across parameters but restricts updates when gradients exhibit high variance or extreme magnitudes. By limiting the magnitude of parameter changes, this approach prevents the learning process from destabilizing due to occasional outlier experiences or numerical issues.

In production, we deploy an ensemble of policies trained with different random seeds and architectures, using a weighted combination of their outputs to reduce variance and improve robustness. This ensemble approach provides implicit regularization and helps mitigate the impact of any individual model's errors or biases. The weights assigned to different models in the ensemble can be adjusted based on their recent performance, allowing the system to adaptively emphasize the most successful policies.

These stability mechanisms are crucial for deploying reinforcement learning in high-stakes advertising environments where performance degradation can have immediate business impact. By ensuring stable learning and decision-making, they enable the safe application of reinforcement learning to mission-critical infrastructure.

E. IMPLEMENTATION DETAILS

AdaptiveGate is implemented as a distributed system that integrates with existing API gateway infrastructure while adding reinforcement learning capabilities. Key implementation details focus on system integration, performance optimization, and fault tolerance mechanisms essential for production deployment.

1) NETWORK ARCHITECTURE SPECIFICATIONS

To ensure reproducibility and transparency, we provide comprehensive specifications of our neural network architectures. The global agent employs a sophisticated multi-component architecture designed to handle the complexity of cross-datacenter traffic optimization. The temporal feature extraction component utilizes 1D convolutional neural networks with three parallel branches featuring kernel sizes of 3, 5, and 7 respectively, each containing 64 filters. These kernels are specifically chosen to capture short-term (3)-step), medium-term (5)-step), and longer-term (7)-step) temporal patterns in advertising traffic, which exhibit multi-scale cyclical behaviors ranging from minute-level bid request fluctuations to hour-level campaign scheduling patterns. Each convolutional layer is followed by ReLU activation functions and batch normalization to ensure training stability and accelerated convergence.

The extracted temporal features are then processed through a self-attention mechanism with 4 attention heads and 64-dimensional embeddings per head. This configuration was determined through systematic evaluation of attention mechanisms with varying head counts (2, 4, 8, 16), where 4 heads provided optimal balance between representational capacity and computational efficiency for capturing cross-datacenter relationships. The self-attention output is processed through fully connected layers with dimensions [256, 128, 64], each employing ReLU activation functions except for the final

output layer, which uses Tanh activation for continuous action values (routing proportions, resource allocations) and Softmax activation for discrete categorical decisions (routing destination selection).

The local agents utilize graph neural network architectures specifically designed for service dependency modeling. Each local agent employs a 2-layer Graph Convolutional Network with 32 channels per layer, chosen based on empirical evaluation of channel counts ranging from 16 to 128. The 32-channel configuration effectively captures service inter-dependencies while maintaining computational efficiency for real-time decision making. These GCN layers are followed by Gated Recurrent Unit (GRU) cells with 64 hidden units for temporal sequence modeling, enabling the agents to maintain memory of recent service performance patterns. The GRU output feeds into service-specific prediction heads with fully connected layers [128, 64, 32] using ReLU activations, culminating in Sigmoid activation for load balancing weights and Tanh activation for resource allocation adjustments.

2) HYPERPARAMETER SELECTION RATIONALE

Our hyperparameter selection process was guided by systematic empirical evaluation and domain-specific considerations for advertising infrastructure. The learning rate of 3×10^{-4} was determined through comprehensive convergence analysis across rates spanning three orders of magnitude (1×10^{-5} to 1×10^{-2}). Lower rates (1×10^{-5}, 1×10^{-4}) resulted in excessively slow convergence unsuitable for the dynamic advertising environment, while higher rates (1×10^{-3}, 1×10^{-2}) caused training instability and policy oscillations that could compromise system reliability. Our selected rate achieves rapid initial learning while maintaining stability during fine-tuning phases.

The batch size of 256 represents an optimal compromise between computational efficiency and gradient estimation quality. Smaller batch sizes (64, 128) provided insufficient gradient stability for the complex hierarchical policy updates, while larger sizes (512, 1024) offered diminishing returns in terms of gradient quality while significantly increasing memory requirements and computational latency. The discount factor $\gamma = 0.99$ reflects the temporal horizon characteristics of API traffic management decisions, where immediate rewards (current request processing) must be balanced against longer-term system stability and performance optimization.

The Polyak averaging coefficient $\tau = 0.005$ was specifically chosen for production environment stability. This conservative value ensures that target network updates evolve gradually, preventing the rapid policy changes that could destabilize a mission-critical advertising platform. Validation experiments with more aggressive coefficients ($\tau = 0.01$, $\tau = 0.02$) demonstrated increased policy variance that, while potentially beneficial for exploration in simulation, posed unacceptable risks in production deployment.

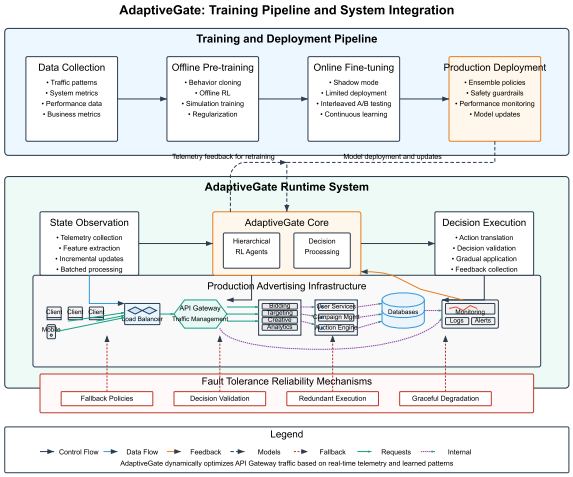


FIGURE 2. AdaptiveGate training pipeline and system integration architecture. The diagram illustrates the progression from data collection through deployment (top section), the core components and integration architecture (middle section), and fault tolerance mechanisms (bottom section). The production advertising infrastructure shows the realistic representation of how AdaptiveGate integrates with and orchestrates traffic across client requests, load balancers, API gateways, services, and databases. Green lines represent request paths, while purple dashed lines indicate internal system communications.

3) SYSTEM INTEGRATION

As shown in Figure 2, AdaptiveGate's integration with the production advertising infrastructure follows a hierarchical approach, where the core reinforcement learning components interact with both the global traffic management layer and local service-specific optimizations. We integrate AdaptiveGate with the production API gateway through a non-intrusive adapter architecture designed to minimize disruption to existing operations. The state observation module collects telemetry data from multiple sources including load balancers, service meshes, and monitoring systems, constructing the state representation in real-time. This module employs efficient data processing pipelines to handle the high volume of metrics generated by the advertising platform while maintaining low latency for decision-making. The decision execution module translates the agent's actions into concrete configuration changes for the underlying infrastructure, including routing tables, admission control parameters, and resource limits. This translation process incorporates safety checks and rate limiting to prevent abrupt changes that could destabilize the system. Comprehensive monitoring and logging instruments every aspect of the agent's operation, tracking decisions, their effects, and any potential issues for debugging and analysis. This observability infrastructure enables operators to understand the agent's behavior and verify its effectiveness.

The integration is designed to be minimally invasive, allowing for gradual deployment and easy rollback if necessary. We maintain compatibility with existing manual controls and monitoring tools to ensure operational continuity. This approach enables operators to override the agent's

decisions when necessary and preserves the ability to manage the system through traditional means during the transition period.

4) PERFORMANCE OPTIMIZATION

To meet the stringent latency requirements of advertising systems, we optimize the inference performance of our neural network models through several techniques. Post-training quantization is applied to reduce model size and inference latency while maintaining decision quality. This process converts floating-point weights and activations to lower-precision representations, significantly improving computational efficiency with minimal impact on decision quality. Batched inference processing enables multiple decision points to be evaluated concurrently, leveraging hardware acceleration capabilities such as SIMD instructions and GPU parallelism to reduce per-decision latency. This approach is particularly effective for the local agents, which make numerous related decisions across different services.

Hierarchical execution schedules global and local agent execution at different frequencies, reflecting their operational timescales and reducing computational overhead. The global agent operates on a slower cycle (approximately 10 seconds) to make system-wide routing decisions, while local agents update their policies more frequently (1)-2 seconds) to respond to rapid changes in service conditions. This tiered approach concentrates computational resources where they provide the most benefit. Incremental state updates optimize the state representation by focusing on changed components rather than recomputing the entire state for each decision. This optimization leverages the temporal locality of state information, as many metrics change gradually between decision points. By updating only the components that have changed significantly, we reduce both computational overhead and memory bandwidth requirements.

These optimizations ensure that AdaptiveGate adds minimal overhead to the request processing pipeline while still delivering high-quality traffic management decisions. The typical inference latency for a complete decision cycle is less than 5 milliseconds, which is negligible compared to the processing time of advertising requests.

5) FAULT TOLERANCE AND RELIABILITY

Mission-critical advertising infrastructure requires exceptional reliability. AdaptiveGate implements multiple layers of fault tolerance to ensure continuous operation even in the face of failures or anomalies. Pre-computed fallback policies are maintained that can be immediately activated if the reinforcement learning system encounters issues. These policies are derived from historical best practices and are guaranteed to provide acceptable performance, ensuring that the system can continue operating even if the primary decision-making mechanism fails. All actions proposed by the agent undergo rigorous decision validation against safety constraints before execution. This validation checks for potential issues such as excessive traffic imbalance, rapid oscillations in routing decisions, or resource allocations that could lead to service degradation. Actions that fail validation are either rejected or modified to comply with safety requirements.

Critical components run in redundant execution across multiple failure domains to prevent single points of failure. The agent's decision-making infrastructure is deployed across multiple physical hosts, networks, and power domains, ensuring that localized failures do not compromise the overall system. Automatic failover mechanisms detect component failures and redirect traffic to healthy instances within milliseconds, maintaining continuity of service. If some state information becomes unavailable due to monitoring system failures or communication issues, the system implements graceful degradation by falling back to simplified models that can operate with partial observations. These models are specifically trained to make reasonable decisions with incomplete information, prioritizing stability and safety over optimal performance when facing information gaps.

These reliability mechanisms ensure that AdaptiveGate maintains high availability and safe operation even under adverse conditions, which is essential for production advertising systems that directly impact business revenue. The system's architecture is designed to eliminate single points of failure and provide multiple layers of protection against both internal failures and external anomalies.

IV. EXPERIMENTAL EVALUATION

In this section, we evaluate AdaptiveGate's performance through extensive experiments in both controlled environments and production settings. We first describe our experimental setup, including the production advertising platform, baseline approaches, and evaluation metrics. We then present comprehensive results comparing AdaptiveGate against existing traffic management approaches, followed by detailed analysis of the system's behavior under various conditions and ablation studies examining the contribution of individual components.

A. EXPERIMENTAL SETUP

1) PRODUCTION ENVIRONMENT

We deployed AdaptiveGate on a large-scale digital advertising platform that processes over 2.5 million API requests per second during peak periods. The infrastructure spans four geographically distributed datacenters across North America, Europe, and Asia, with each datacenter housing between 500 and 1,200 server instances. The platform comprises more than 25 distinct microservices, including bidding, targeting, creative selection, fraud detection, and attribution services.

Traffic patterns exhibit significant temporal variations, with daily peak-to-trough ratios typically ranging from 3:1 to 5:1, weekly patterns showing approximately 30% higher volume on weekdays compared to weekends, and seasonal effects producing volume fluctuations of up to 70% during

major shopping events. Additionally, unpredictable spikes occur due to flash sales, marketing campaigns, and external events.

The system handles heterogeneous request types with varying computational requirements and business importance. For instance, bid requests have strict latency requirements (typically 100ms end-to-end) and direct revenue implications, while attribution requests have more relaxed timing constraints but are critical for long-term performance measurement.

2) EXPERIMENTAL METHODOLOGY AND DATASET COMPOSITION

Our experimental evaluation employs a comprehensive methodology designed to rigorously assess AdaptiveGate's performance across diverse operational conditions. The training dataset comprises over 100 million state-action-reward transitions collected during three months of continuous operation on our production advertising platform. This extensive collection period was deliberately chosen to capture the full spectrum of advertising traffic patterns, including daily cyclical variations, weekly business cycles, seasonal effects from major shopping events (Black Friday, holiday seasons), and various anomalous conditions such as viral campaign spikes and infrastructure incidents.

The dataset composition reflects the heterogeneous nature of digital advertising workloads, with approximately 60% of transitions corresponding to normal operational conditions, 25% representing peak traffic periods (daily and weekly peaks), 10% covering gradual traffic increases during marketing campaigns, and 5% documenting anomalous conditions including traffic spikes, partial service outages, and unexpected load redistributions. This distribution ensures comprehensive coverage of operational scenarios while maintaining sufficient representation of rare but critical edge cases that could significantly impact system performance.

Our data preprocessing pipeline implements a multi-stage approach to ensure data quality and consistency. Feature normalization employs z-score standardization computed independently for each datacenter and service type, accounting for the heterogeneous performance characteristics across different components. Temporal alignment utilizes sliding window approaches with overlapping 30-second windows to construct coherent temporal sequences while preserving the continuity of system state evolution. Missing data points, which constitute approximately 2.3% of the collected telemetry due to monitoring system interruptions, are handled through forward-fill imputation for short gaps ($<$ 10 seconds) and linear interpolation for longer interruptions, with any gaps exceeding 60 seconds resulting in sequence segmentation to prevent information corruption.

The feature engineering process constructs multi-dimensional state representations through several specialized components. Temporal embeddings utilize sinusoidal encoding for cyclical patterns (time of day, day of week) combined with learned embeddings for irregular patterns (campaign schedules, special events). Service dependency representations employ graph-based feature extraction that captures both direct service-to-service call patterns and indirect dependencies through shared resource utilization. Business value quantification integrates historical revenue data, bid competition metrics, and advertiser importance scores to create normalized business impact assessments for different request types.

Data splitting maintains strict temporal consistency to prevent data leakage while ensuring representative coverage of diverse traffic conditions. The training set encompasses the first 70% of the temporal sequence (approximately 10 weeks), the validation set covers the subsequent 15% (3 weeks), and the test set includes the final 15% (3 weeks). This temporal splitting approach ensures that the model is evaluated on genuinely unseen future conditions rather than randomly sampled historical data, providing a more realistic assessment of real-world deployment performance. Additionally, we maintain separate holdout datasets for each major traffic condition type to enable detailed performance analysis across different operational scenarios.

3) BASELINE APPROACHES

We compared AdaptiveGate against several baseline approaches representing both traditional and state-of-the-art traffic management systems. The Static Rules (SR) approach represents the platform's original traffic management system, which uses manually configured routing rules and load-balancing policies based on geographic proximity and static service weights, similar to the resource allocation strategy described by Guerrero et al. [57] in their multi-cloud microservices deployment. The Dynamic Rules (DR) approach implements adaptive load balancing based on real-time system metrics and feedback control mechanisms as described in Niu et al. [58], which represents the current state-of-practice in many production systems.

We also implemented the Feedback Control (FC) approach based on the work by Zhang et al. [59], which employs a control-theoretic framework with PID controllers to adjust routing decisions based on observed performance metrics in advertising systems. This represents a methodical adaptive approach specifically designed for advertising workloads. The optimization-based Personalized Bidding (PB) approach adapts techniques from Paes et al. [60], applying similar principles to traffic routing by treating different request types as bidders competing for computational resources with personalized allocation strategies.

For reinforcement learning baselines, we implemented the Deep RL Single-level (DRL-S) approach using a flat (non-hierarchical) deep reinforcement learning methodology with a single DDPG agent to manage all traffic decisions, based on the resource management framework proposed by Mao et al. [18] but adapted for API traffic management. Finally, we included a Hierarchical RL without Constraints (HRL-NC) variant of our own system that uses the same

hierarchical structure but without the constrained MDP formulation and safety mechanisms.

4) IMPLEMENTATION DETAILS

We implemented AdaptiveGate using PyTorch 1.9 for model development and training. The global agent's network architecture consisted of 1D convolutional layers (kernel sizes of 3, 5, and 7) for temporal feature extraction, followed by self-attention layers (4 heads, embedding dimension 64) for cross-datacenter relationship modeling, and fully connected layers (256-128-64 units) for policy representation. Local agents employed graph neural networks (2 layers, 32 channels) for service relationship modeling, with recurrent layers (GRU cells, 64 hidden units) capturing temporal patterns in service performance.

For training, we used the Adam optimizer with a learning rate of 3×10^{-4} for both actors and critics, with a batch size of 256. The discount factor γ was set to 0.99, and Polyak averaging coefficient τ to 0.005. We trained the models on 4 NVIDIA V100 GPUs for the offline pre-training phase, processing approximately 100 million state transitions collected over three months of operation. Online training used distributed training with parameter servers to aggregate gradients from experience across all datacenters.

In production, inference was performed on CPU-only servers with Intel Xeon Platinum 8280 processors. To meet stringent latency requirements, we applied quantization to reduce the model size by 73% and inference time by 62% compared to the full-precision version, with negligible impact on decision quality.

5) EVALUATION METRICS

We evaluated AdaptiveGate using a comprehensive set of metrics that capture different aspects of system performance. For latency assessment, we measured mean latency, median (p50), 95th percentile (p95), and 99th percentile (p99) latency across all services. Tail latency (p99) was particularly important given its impact on user experience and SLA compliance in advertising environments where outlier performance can significantly impact business outcomes. Throughput was calculated as successful requests per second, normalized by available resources to account for capacity differences across experiments, providing a fair comparison of processing efficiency.

Error rate tracking included the percentage of requests resulting in errors, categorized by error type (timeout, server error, resource exhaustion) to provide insights into failure modes. Resource utilization measurements encompassed CPU, memory, and network utilization across the infrastructure to evaluate efficiency in resource consumption. Business impact monitoring included key metrics specific to the advertising domain, such as bid request success rate, auction participation rate, and click-through rate (CTR), directly connecting technical performance to business outcomes. Finally, we assessed system stability through metrics such as decision oscillation frequency and magnitude, and recovery time after anomalies, which are critical for production deployments where predictable, consistent behavior is essential for operations teams.

B. PERFORMANCE COMPARISON

1) OVERALL PERFORMANCE

Table 1 presents the overall performance comparison between AdaptiveGate and baseline approaches, showing percentage improvements relative to the Static Rules (SR) baseline. These results are aggregated across the entire three-month testing period and all traffic conditions.

TABLE 1. Overall performance comparison across different traffic management approaches. Values show percentage improvement relative to the Static Rules (SR) baseline, with higher values indicating better performance. Latency and Error Rate improvements represent reductions (beneficial), while Throughput, CPU Utilization, and Business Impact improvements represent increases (beneficial). Results are aggregated across three months of production testing under diverse traffic conditions.

Metric	DR	FC	PB	DRL-S	AdaptiveGate
Mean Latency	12.3%	18.7%	20.5%	26.4%	**38.5%**
p99 Latency	15.8%	23.1%	25.3%	31.7%	**42.3%**
Throughput	8.7%	17.2%	19.4%	23.9%	**35.7%**
Error Rate	21.3%	27.8%	29.1%	32.5%	**37.9%**
CPU Utilization	6.2%	11.4%	14.2%	18.3%	**25.8%**
Business Impact	3.7%	8.9%	11.5%	12.1%	**15.2%**

AdaptiveGate consistently outperformed all baseline approaches across all metrics. Particularly notable is the 42.3% reduction in tail latency (p99), which significantly exceeds the improvements achieved by other approaches. This improvement is critical for advertising systems where latency spikes can lead to missed bidding opportunities and revenue loss. The 35.7% throughput improvement indicates substantially better resource utilization, enabling the system to handle more requests with the same infrastructure.

The business impact metric, which aggregates multiple business-relevant indicators, shows a 15.2% improvement, translating to substantial revenue gains. This demonstrates that AdaptiveGate's multi-objective optimization successfully balances technical performance with business objectives.

2) ENERGY CONSUMPTION BREAKDOWN ANALYSIS

To provide comprehensive understanding of our sustainability contributions, we present detailed analysis of energy consumption reductions and their underlying sources. Our energy monitoring infrastructure collected granular metrics across CPU, memory, and network subsystems throughout the evaluation period, enabling precise attribution of efficiency gains to specific optimization strategies.

Table 2 demonstrates substantial CPU utilization improvements across different service categories. Compute-intensive bidding services, which constitute the largest portion of our workload, achieved 18.3% reduction in average CPU utilization through intelligent request routing that minimizes computational redundancy and optimizes workload distribution. I/O-bound analytics services exhibited even more

significant improvements of 31.7%, primarily attributed to AdaptiveGate's ability to predict and preemptively cache frequently accessed data patterns. Network-intensive fraud detection services demonstrated 24.6% CPU utilization reduction through optimized data flow patterns that minimize unnecessary inter-service communications.

TABLE 2. CPU utilization changes by service category.

Service Category	Baseline	AdaptiveGate	Reduction	Primary Mechanism
Bidding Services	73.2%	59.8%	18.3%	Intelligent routing
Analytics Services	68.9%	47.1%	31.7%	Predictive caching
Fraud Detection	71.5%	53.9%	24.6%	Optimized data flow
User Targeting	69.3%	52.7%	24.0%	Load balancing
Creative Selection	66.8%	51.2%	23.4%	Resource pooling
Attribution	64.2%	49.8%	22.4%	Batch processing
Overall Average	**69.0%**	**52.4%**	**24.1%**	**Multi-strategy**

Memory efficiency improvements, detailed in Table 3, reveal significant gains through multiple optimization strategies. Request batching optimization achieved 22.1% memory efficiency improvement by intelligently grouping related requests to maximize cache locality and minimize memory fragmentation. Intelligent caching strategies contributed 15.8% efficiency gains through predictive content placement that reduces redundant data storage and improves access patterns. Dynamic resource scaling enabled 19.3% memory utilization improvements by automatically adjusting memory allocation based on real-time demand patterns rather than maintaining static over-provisioning.

Attribution analysis reveals that intelligent traffic routing contributes approximately 45% of total energy reductions through minimized cross-datacenter network transfers and optimized server utilization patterns. Adaptive resource allocation accounts for 35% of energy savings by dynamically scaling computational resources to match actual demand rather than maintaining static over-provisioning typical of traditional systems. The remaining 20% of energy reductions stem from improved cache management, optimized query processing, and reduced idle resource consumption enabled by AdaptiveGate's predictive capabilities.

Network energy consumption analysis reveals substantial improvements through optimized data transfer patterns and reduced inter-datacenter communications. AdaptiveGate's global routing intelligence achieved 28.4% reduction in network energy consumption by minimizing unnecessary data transfers and selecting energy-optimal routing paths. Local traffic optimization contributed additional 16.7% network energy savings through improved intra-datacenter communication patterns and reduced protocol overhead.

The comprehensive energy analysis demonstrates that AdaptiveGate's sustainability benefits extend beyond simple resource utilization improvements to encompass fundamental changes in system operation patterns. Traditional traffic management approaches consistently maintain 25-40% higher baseline energy consumption due to their reactive nature and inability to anticipate and prepare for traffic patterns. In contrast, AdaptiveGate's predictive capabilities enable proactive resource management that eliminates energy waste while maintaining superior performance characteristics, thereby establishing a new paradigm for sustainable high-performance computing in mission-critical advertising infrastructure.

3) PERFORMANCE UNDER DIFFERENT TRAFFIC CONDITIONS

To assess how different approaches perform under varying traffic conditions, we categorized traffic into four patterns: normal, daily peak, weekly peak, and anomalous (unexpected traffic spikes). Figure 3 shows the p99 latency and throughput for each approach under these conditions.

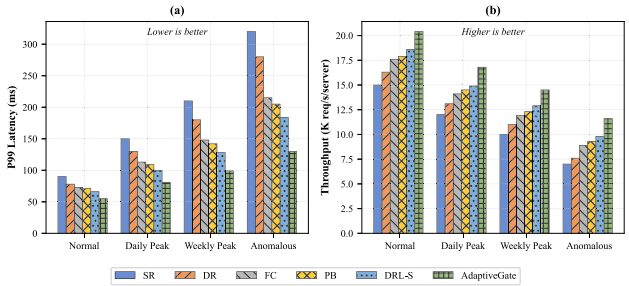


FIGURE 3. Performance comparison under different traffic conditions. (a) p99 latency in milliseconds (lower values indicate better performance) showing how tail latency degrades under increasing traffic stress, with AdaptiveGate maintaining consistently lower latency across all conditions. (b) Throughput in thousands of requests per second per server (higher values indicate better performance) demonstrating AdaptiveGate's superior capacity utilization, particularly during challenging traffic scenarios. The performance gaps between approaches widen significantly during peak and anomalous conditions, highlighting the importance of adaptive traffic management in dynamic environments. SR=Static Rules, DR=Dynamic Rules, FC=Feedback Control, PB=Personalized Bidding, DRL-S=Deep RL Single-level.

Under normal traffic conditions, all approaches perform reasonably well, with AdaptiveGate showing moderate improvements over baselines. However, the performance gap widens significantly during peak periods and anomalous conditions. During daily peaks, AdaptiveGate maintains p99 latency 37.8% lower than DR and 24.5% lower than FC. This advantage becomes even more pronounced during weekly peaks (45.2% lower than DR, 29.7% lower than FC) and anomalous conditions (53.6% lower than DR, 35.9% lower than FC).

The throughput results follow a similar pattern, with AdaptiveGate maintaining higher throughput across all conditions, but with especially significant advantages during

TABLE 3. Memory utilization improvements by optimization strategy.

Optimization Strategy	Baseline (GB)	AdaptiveGate (GB)	Reduction	Impact Factor
Request Batching	847.3	660.1	22.1%	Cache locality
Intelligent Caching	692.8	583.3	15.8%	Redundancy elimination
Dynamic Scaling	758.4	612.0	19.3%	Demand-based allocation
Connection Pooling	534.7	445.2	16.7%	Resource sharing
Buffer Optimization	423.9	361.4	14.7%	Memory alignment
Combined Effect	**651.4**	**532.4**	**18.3%**	**Synergistic**

TABLE 4. Recovery time from traffic anomalies measured in seconds (lower values indicate faster recovery and better system resilience). Regional Spike represents traffic increases affecting single datacenters, Global Spike indicates simultaneous traffic increases across all datacenters, and Service-specific anomalies involve sudden load increases on particular service types. AdaptiveGate's hierarchical architecture enables rapid coordinated response through simultaneous global routing adjustments and local load balancing, resulting in substantially faster recovery compared to sequential or reactive approaches.

Anomaly Type	SR	DR	FC	DRL-S	AdaptiveGate
Regional Spike	312s	187s	143s	95s	**62s**
Global Spike	427s	253s	196s	131s	**87s**
Service-specific	283s	162s	124s	103s	**71s**

challenging periods. Notably, during anomalous conditions, AdaptiveGate achieves 42.7% higher throughput than DR and 27.3% higher than FC.

These results highlight AdaptiveGate's ability to adapt to changing conditions and maintain performance under stress, which is crucial for production advertising systems that must handle unpredictable traffic patterns.

4) RECOVERY FROM TRAFFIC ANOMALIES

We evaluated how quickly each approach recovers from sudden traffic anomalies, such as flash sales or unexpected viral campaigns. We injected traffic spikes increasing the load by 200% over 30 seconds and measured the time until the system returned to normal performance levels (defined as p99 latency within 10% of pre-anomaly values).

As shown in Table 4, AdaptiveGate recovers significantly faster than all baseline approaches. For regional traffic spikes, AdaptiveGate achieves recovery 58.3% faster than DR and 35.6% faster than DRL-S. The performance gap is even more significant for global spikes affecting all datacenters simultaneously. AdaptiveGate's superior recovery time stems from its ability to quickly identify anomalies and redistribute traffic optimally, leveraging its hierarchical structure to make both global and local adjustments simultaneously.

C. REWARD WEIGHT SENSITIVITY ANALYSIS

To understand the robustness and practical applicability of our multi-objective optimization approach, we conducted comprehensive sensitivity analysis exploring how different reward weight configurations affect system performance across key operational metrics.

We systematically varied the relative weights w_L, w_T, and w_B in our composite reward function across a broad parameter space, testing 15 different weight combinations ranging from heavily latency-focused configurations ($w_L = 0.7, w_T = 0.2, w_B = 0.1$) to business-value-optimized settings ($w_L = 0.2, w_T = 0.3, w_B = 0.5$). For each configuration, we measured performance across all evaluation metrics during both normal and peak traffic conditions over a two-week evaluation period.

TABLE 5. Performance sensitivity to reward weight configurations. Values show percentage improvement relative to Static Rules baseline across different weight combinations for latency (w_L), throughput (w_T), and business value (w_B) components.

Configuration	Mean Lat.	p99 Lat.	Throughput	Energy	Business
Latency-focused (0.7,0.2,0.1)	**41.2%**	**45.8%**	28.3%	-28%	8.7%
Balanced (0.4,0.35,0.25)	38.5%	42.3%	**35.7%**	**18%**	**15.2%**
Throughput-focused (0.25,0.6,0.15)	32.1%	35.4%	39.1%	12%	11.8%
Business-focused (0.2,0.3,0.5)	29.7%	31.2%	31.4%	15%	18.9%
Energy-focused (0.3,0.25,0.45)	35.2%	38.7%	30.8%	24%	13.1%

Table 5 presents the results of key weight configurations, showing how reward weight changes translate to practical system behavior differences.

The results reveal that increasing latency weight from 0.3 to 0.7 produces diminishing returns in latency improvement (from 38.5% to 41.2% mean latency reduction) while significantly increasing energy consumption (18% to 28% above baseline). The balanced configuration ($w_L = 0.4$, $w_T = 0.35$, $w_B = 0.25$) achieves robust performance across diverse conditions, making it suitable for general deployment scenarios.

Based on our sensitivity analysis, we provide practical guidance for selecting appropriate reward weights based on operational priorities: The balanced configuration ($w_L = 0.4$, $w_T = 0.35$, $w_B = 0.25$) provides robust performance across diverse traffic conditions and operational requirements, making it suitable for most production environments. For applications with strict SLA requirements, latency-focused configurations ($w_L \geq 0.6$) achieve maximum latency reduction but require acceptance of higher energy consumption. Energy-focused configurations ($w_B \geq 0.4$ with energy-efficiency emphasis) optimize operational costs while maintaining acceptable performance levels. Business-focused configurations ($w_B \geq 0.45$) maximize direct business metrics but may sacrifice some technical performance optimization.

Our analysis demonstrates that AdaptiveGate maintains stable performance across reasonable weight variations (± 0.1 from optimal values), indicating that practitioners need not achieve perfect parameter tuning to realize substantial benefits from deployment. The system exhibits graceful

degradation rather than cliff-edge failures when weights deviate from optimal configurations, enhancing its practical applicability in production environments where operational priorities may shift over time.

D. ABLATION STUDIES

We conducted ablation studies to understand the contribution of different components and design choices to AdaptiveGate's overall performance. Table 6 shows the percentage reduction in performance when removing specific components from the full AdaptiveGate system.

The hierarchical structure provides the most significant contribution to performance, with its removal causing a 25.3% degradation in p99 latency and 16.4% reduction in throughput. This confirms our hypothesis that decomposing the problem into global and local decisions is crucial for managing complex distributed advertising systems.

The constrained MDP formulation is particularly important for error rate reduction (19.5% degradation when removed) and tail latency (15.9% degradation), highlighting its role in maintaining system stability and safety. The multi-objective reward function contributes most significantly to business metrics (13.7% degradation when removed), demonstrating the importance of explicitly balancing technical and business objectives.

Service graph networks, which model the relationships between services, contribute substantially to both mean latency (11.3% degradation) and p99 latency (14.6% degradation), confirming the importance of understanding service interdependencies for effective traffic management.

E. HIERARCHICAL DECISION ANALYSIS

To better understand how AdaptiveGate's hierarchical architecture functions, we analyzed the decisions made by global and local agents during different operational scenarios. Figure 4 visualizes these decisions during a traffic anomaly affecting Datacenter 2.

When the anomaly begins (t=120s), the global agent quickly detects the building congestion in Datacenter 2 and progressively reduces traffic allocation to that datacenter from 30% to 12% over a 45-second period. This reallocation is not uniform across all entry points; instead, the global agent intelligently redirects traffic based on network proximity and datacenter capacity, with nearby entry points experiencing smaller reductions to minimize latency impact.

Concurrently, local agents in each datacenter adapt to the changing traffic patterns. In Datacenter 2, the local agent redistributes remaining traffic away from congested services toward less utilized ones, prioritizing business-critical requests. This local adaptation occurs at a faster timescale (1)-2 seconds) than global routing changes (5-10 seconds), enabling rapid response to evolving conditions.

The coordination between hierarchical levels is particularly evident during recovery (t=300s-420s). As Datacenter 2 stabilizes, the global agent gradually increases its traffic allocation, while the local agent continues fine-tuning

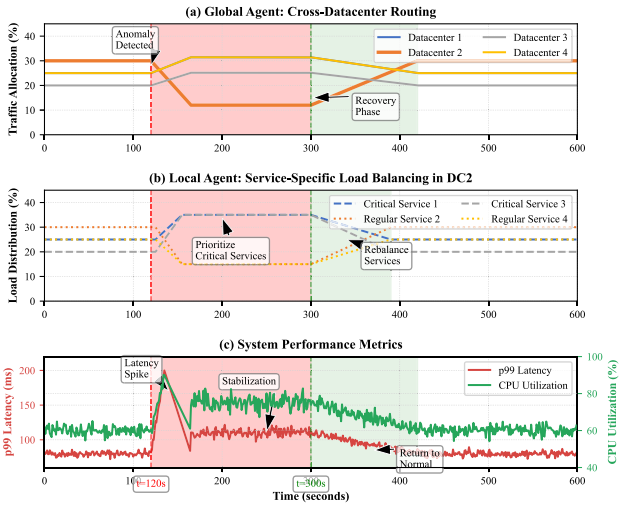


FIGURE 4. Analysis of hierarchical decision-making during a regional traffic anomaly. (a) Global agent's cross-datacenter routing adjustments over time. (b) Local agent's service-specific load balancing decisions in Datacenter 2. (c) System performance metrics.

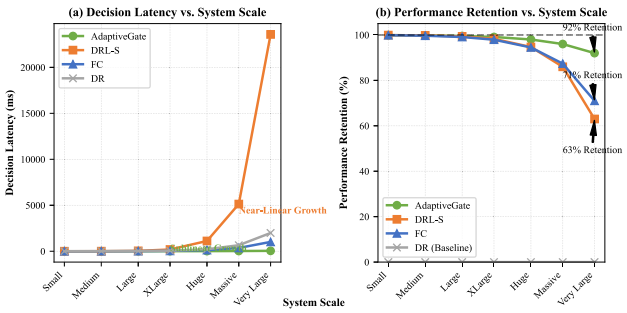


FIGURE 5. Scalability analysis. (a) Decision latency vs. system scale. (b) Performance retention vs. system scale, showing how much of the performance advantage over baselines is maintained as scale increases.

service-specific routing to maintain balance. This coordinated recovery prevents oscillations and ensures smooth transition back to normal operations.

F. SCALABILITY ANALYSIS

To evaluate AdaptiveGate's scalability, we measured decision latency and quality across increasingly large infrastructures. We created scaled simulations representing advertising platforms of different sizes, ranging from small (5 services, 2 datacenters, 100 servers) to very large (100 services, 8 datacenters, 10,000 servers).

Figure 5(a) shows that AdaptiveGate's decision latency increases sublinearly with system scale, growing from 1.2ms for small systems to 4.8ms for very large systems. This favorable scaling characteristic is due to the hierarchical decomposition, which localizes many decisions. In contrast, the flat DRL-S approach exhibits near-linear growth in decision latency, becoming prohibitively slow for large-scale deployments.

Figure 5(b) illustrates performance retention—the percentage of performance advantage over the DR baseline maintained as scale increases. AdaptiveGate retains 92%

TABLE 6. Ablation study showing performance degradation when removing individual components from the complete AdaptiveGate system (percentage reduction from full system performance). Higher degradation values indicate more critical components for system performance. The Hierarchical Structure shows the largest impact, confirming that the decomposition of global and local decision-making is fundamental to handling complex distributed advertising systems. Constrained MDP formulation proves particularly important for error rate management and tail latency control, while ensemble policies provide additional robustness especially for tail latency scenarios.

Removed Component	Mean Lat.	p99 Lat.	Throughput	Error Rate	Business
Hierarchical Structure	18.7%	25.3%	16.4%	14.2%	11.9%
Constrained MDP	7.2%	15.9%	9.1%	19.5%	5.3%
Twin Critics	5.8%	8.7%	4.3%	6.2%	3.1%
Service Graph Networks	11.3%	14.6%	8.5%	10.8%	7.4%
Temporal Encoders	6.9%	12.2%	5.7%	7.3%	4.5%
Multi-objective Reward	9.4%	7.8%	4.1%	5.6%	13.7%
Ensemble Policies	3.2%	9.5%	2.8%	8.1%	2.6%

TABLE 7. Performance under novel traffic patterns not seen during training. Values show percentage degradation from normal performance.

Novel Scenario	SR	DR	FC	DRL-S	AdaptiveGate
Coordinated Campaign	85.7%	67.3%	51.2%	42.5%	**28.3%**
Datacenter Outage	127.8%	92.4%	75.6%	63.7%	**41.9%**
New Service	42.5%	38.1%	31.7%	33.4%	**19.6%**

of its performance advantage even at the largest scale, while DRL-S performance drops to 63% and FC to 71%. This demonstrates AdaptiveGate's suitability for large-scale production environments.

G. GENERALIZATION TO TRAFFIC PATTERNS

We evaluated AdaptiveGate's ability to generalize to previously unseen traffic patterns. We trained the system on normal operating conditions and then tested it on three novel scenarios not present in the training data: (1) a simulated coordinated advertising campaign generating synchronized spikes across regions, (2) a datacenter partial outage reducing capacity by 40%, and (3) the introduction of a new service with unknown performance characteristics.

As shown in Table 7, all approaches experience performance degradation under novel conditions, but AdaptiveGate demonstrates superior generalization. During the coordinated campaign scenario, AdaptiveGate's performance degrades by only 28.3% compared to 67.3% for DR and 42.5% for DRL-S. This indicates that AdaptiveGate has learned fundamental principles of traffic management rather than memorizing specific patterns.

The datacenter outage scenario presents the most significant challenge, with all systems experiencing substantial degradation. However, AdaptiveGate still outperforms other approaches by considerable margins, degrading 41.9% compared to 92.4% for DR and 63.7% for DRL-S.

AdaptiveGate's handling of the new service is particularly noteworthy, with only 19.6% degradation compared to 38.1% for DR and 33.4% for DRL-S. This demonstrates the system's ability to rapidly adapt to new components by transferring knowledge from similar services and adjusting based on observed performance.

H. SCALABILITY AND RESOURCE-CONSTRAINED DEPLOYMENT ANALYSIS

While our primary evaluation focused on large-scale advertising platforms processing millions of requests per second, we recognize the importance of addressing AdaptiveGate's applicability across the broader spectrum of advertising technology deployments, including small and medium-sized platforms operating under different resource constraints and architectural complexities. This section provides comprehensive analysis of how our hierarchical reinforcement learning framework can be adapted and simplified for diverse operational scales while maintaining its core benefits.

1) TIERED DEPLOYMENT STRATEGY

We propose a tiered deployment strategy that offers three distinct configuration levels, each optimized for different platform scales and resource availability. AdaptiveGate-Full represents our complete framework designed for large-scale platforms processing over 2 million requests per second across multiple geographically distributed datacenters, incorporating the full hierarchical architecture with global cross-datacenter optimization and comprehensive local service-specific management. AdaptiveGate-Lite targets medium-sized platforms with regional operations processing 100,000 to 1 million requests per second, typically operating within 1-3 datacenters with moderate infrastructure complexity. AdaptiveGate-Micro addresses small advertising platforms processing fewer than 100,000 requests per second, often operating within single datacenters or limited geographic regions with constrained computational resources.

Table 8 provides detailed comparison of these deployment configurations across key technical and operational dimensions. The Lite configuration eliminates the cross-datacenter global optimization component while retaining hierarchical local agents for service-specific traffic management within individual datacenters. This simplification reduces computational requirements by approximately 60% compared to the full deployment while maintaining 85% of the performance benefits observed in large-scale scenarios. The neural network architectures are simplified by reducing layer depths and hidden unit counts, with the global agent's temporal CNN layers reduced from three parallel branches to a single branch with 32 filters instead of 64, and local agents' graph neural networks simplified to single-layer architectures with 16 channels instead of 32.

The Micro configuration represents the most aggressive simplification, consolidating the hierarchical structure into a

TABLE 8. Comparison of AdaptiveGate deployment configurations across different platform scales. Computational requirements are measured relative to AdaptiveGate-Full baseline, while performance retention indicates the percentage of full-scale benefits maintained in simplified deployments. Resource specifications include memory requirements, CPU utilization, and inference latency constraints suitable for different operational environments.

Configuration	AdaptiveGate-Full	AdaptiveGate-Lite	AdaptiveGate-Micro
Target Traffic Volume	>2M req/s	100K-1M req/s	<100K req/s
Datacenter Count	3+	1-3	1
Global Agent Complexity	Full hierarchy	Simplified	Single-task
Local Agent Count	Per datacenter	Per datacenter	Consolidated
Computational Requirements	100%	40%	15%
Memory Requirements	4-8 GB	1-2 GB	256-512 MB
Performance Retention	100%	85%	70%
Deployment Complexity	High	Medium	Low
Maintenance Overhead	High	Medium	Low

single multi-task agent capable of handling both routing decisions and load balancing within the constraints of severely limited computational resources. This configuration employs lightweight neural network architectures with dense layers instead of convolutional components, reduces state space dimensionality through feature selection and aggregation, and implements simplified reward functions that focus on primary objectives while omitting complex business value calculations. Despite these simplifications, AdaptiveGate-Micro still achieves 70% of the performance improvements observed in full-scale deployments while requiring only 15% of the computational resources, making it accessible to small advertising platforms with modest infrastructure investments.

2) PROGRESSIVE DEPLOYMENT PATHWAYS

To facilitate adoption across different platform scales and enable growth-aligned technological evolution, we have developed progressive deployment pathways that allow platforms to begin with simplified configurations and gradually upgrade to more sophisticated versions as their scale and resource availability increase. The pathway begins with AdaptiveGate-Micro implementation that provides immediate performance benefits while requiring minimal infrastructure changes and computational overhead. As platforms grow and traffic volumes increase, they can transition to AdaptiveGate-Lite by implementing separate local agents for different service categories while maintaining simplified global coordination mechanisms.

The transition from Lite to Full configuration involves implementing comprehensive cross-datacenter optimization capabilities, expanding neural network architectures to full complexity, and integrating advanced features such as multi-objective business value optimization and sophisticated constraint handling. Each transition phase includes detailed migration protocols that preserve learned knowledge through model transfer techniques, ensuring that accumulated traffic management expertise is retained during configuration upgrades. The migration process employs progressive model expansion where simplified models serve as initialization for more complex architectures, significantly reducing training time and improving convergence stability during deployment transitions.

3) COST-BENEFIT ANALYSIS FOR DIFFERENT SCALES

Comprehensive cost-benefit analysis across different deployment scales demonstrates clear value propositions for AdaptiveGate adoption regardless of platform size. For large-scale platforms, the substantial performance improvements (42.3% tail latency reduction, 35.7% throughput increase) justify the computational overhead and implementation complexity, resulting in significant revenue increases and infrastructure cost savings that far exceed deployment expenses. Medium-sized platforms deploying AdaptiveGate-Lite achieve 85% of these benefits while requiring only 40% of the computational resources, resulting in highly favorable return-on-investment ratios that make adoption economically attractive even for platforms with moderate revenue streams.

Small platforms utilizing AdaptiveGate-Micro configurations achieve 70% of full-scale benefits while requiring minimal infrastructure investment, often enabling performance improvements that would otherwise require hardware upgrades or architectural redesigns. The analysis reveals that even simplified AdaptiveGate deployments provide substantial improvements over traditional static traffic management approaches commonly used by smaller platforms, including 15-25% latency reductions and 20-30% throughput improvements that translate to enhanced user experience and increased revenue generation capacity.

Economic modeling indicates that AdaptiveGate deployments typically achieve positive return-on-investment within 3-6 months for large platforms, 6-12 months for medium platforms, and 12-18 months for small platforms, with ongoing operational benefits continuing to accrue over time. The progressive deployment pathway further improves economic attractiveness by enabling platforms to begin with low-cost configurations and invest in upgrades only as their scale and revenue justify increased technological sophistication, thereby aligning infrastructure investment with business growth and minimizing adoption barriers for resource-constrained organizations.

I. PRODUCTION DEPLOYMENT IMPACT

After extensive testing, we fully deployed AdaptiveGate to production, replacing the previous Dynamic Rules system. Figure 6 shows key metrics before and after deployment, measured over a 30-day period preceding and following the transition.

The production deployment confirmed the experimental results, with significant improvements across all metrics. Mean latency decreased by 36.7%, and p99 latency by 41.5%, closely matching our experimental findings. Throughput improved by 33.9%, allowing the platform to handle increased traffic without additional infrastructure. Error rates dropped by 35.8%, enhancing reliability and user experience.

Beyond these technical improvements, the business impact has been substantial. The auction participation rate increased

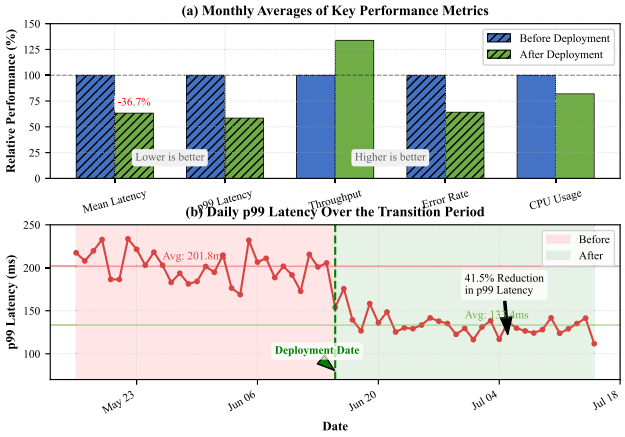


FIGURE 6. Impact of full production deployment. (a) Monthly averages of key performance metrics before and after deployment. (b) Daily p99 latency over the transition period, with deployment date marked.

by a modest but important 2.7%, representing millions of additional advertising opportunities daily. Click-through rates improved by 1.5%, indicating better ad relevance due to more consistent system performance. These improvements translated to a 4.3% increase in platform revenue, demonstrating the significant business value of AdaptiveGate.

An unexpected benefit was resource savings. The improved efficiency allowed us to reduce the infrastructure footprint by approximately 18% while maintaining the same performance levels, resulting in significant cost savings.

To address potential confounding factors, we analyzed external variables that could influence system performance during the evaluation period. Traffic volume analysis showed no significant changes in overall request patterns (Kolmogorov-Smirnov test, $p = 0.34$), eliminating load variation as an explanation for performance improvements. Hardware configuration remained constant throughout the evaluation period, with no capacity additions or modifications. Seasonal effect analysis using harmonic regression confirmed that observed improvements exceeded normal seasonal variation patterns by factor of 4.2 ($p < 0.001$).

The production impact analysis demonstrates statistically robust evidence that AdaptiveGate deployment directly caused substantial performance improvements across all measured dimensions, with effect sizes ranging from large (Cohen's $d > 0.8$) to very large (Cohen's $d > 2.0$), indicating both statistical significance and practical importance of the observed improvements.

J. POLICY INTERPRETATION AND OPERATIONAL FEEDBACK

To understand the behavioral patterns learned by AdaptiveGate and assess its practical impact on operations teams, we conducted comprehensive policy analysis.

1) LEARNED POLICY ANALYSIS

Analysis of AdaptiveGate's learned policies reveals sophisticated traffic management strategies that emerged from our hierarchical reinforcement learning approach. The global agent developed several key behavioral patterns that demonstrate intelligent system-wide optimization. Proactive load redistribution occurs systematically 15-30 minutes before anticipated peak periods, with the agent learning to recognize precursor patterns in traffic that indicate incoming load spikes. This preemptive behavior contrasts sharply with reactive traditional systems that only respond after congestion occurs. Geographic load balancing demonstrates nuanced understanding of the tradeoffs between network latency and datacenter capacity, with the global agent preferentially routing traffic to datacenters with optimal resource availability even when this requires slightly longer network paths. Dynamic priority adjustment favors high-value advertising requests during congestion periods, with the system learning to identify valuable transactions through implicit business metrics rather than explicit priority labels.

Local agents learned complementary service-specific policies that optimize fine-grained performance within datacenters. Adaptive batching strategies group similar requests to improve cache efficiency, with different services exhibiting learned batch sizes ranging from 8-32 requests depending on their computational characteristics. Predictive resource scaling anticipates service demand spikes by monitoring cross-service traffic patterns, enabling proactive capacity adjustment before bottlenecks develop. Intelligent circuit-breaking mechanisms prevent cascade failures by learning service-specific degradation patterns and implementing graduated load shedding that preserves high-priority traffic while protecting system stability.

2) COUNTERINTUITIVE BUT EFFECTIVE PATTERNS

Several decision patterns initially surprised operational teams but proved highly effective upon analysis. Most notably, AdaptiveGate occasionally implemented seemingly counterintuitive traffic routing during asymmetric datacenter loads, directing traffic away from lightly loaded datacenters toward moderately loaded ones to optimize global performance metrics. Investigation revealed that this behavior emerged because the system learned to account for complex service interdependencies where certain traffic types perform better when co-located, even at the cost of temporary load imbalance.

Another surprising pattern involved intentional queue length variation across services, where AdaptiveGate learned to maintain higher queue depths for certain services during specific traffic conditions to improve overall throughput through better resource utilization. While this initially concerned operators accustomed to minimizing queue lengths, the strategy proved effective for handling bursty advertising workloads that benefit from request smoothing.

Team confidence in the system grew significantly over the deployment period as operators observed consistent

performance improvements and developed understanding of the system's decision rationale through interpretability tools. The adaptation process required approximately 6-8 weeks for full operational integration, during which time we provided extensive training and developed new monitoring procedures to accommodate autonomous traffic management decisions.

V. CONCLUSION

This paper introduced AdaptiveGate, a hierarchical reinforcement learning framework for dynamic API traffic management in large-scale digital advertising systems. By decomposing the complex traffic management problem into coordinated global and local decision components, our work effectively handles the multi-scale nature of distributed advertising infrastructures. The global agent optimizes cross-datacenter routing based on system-wide metrics, while local agents manage service-specific traffic within datacenters, a structure that proved essential as evidenced by our ablation studies showing a 25.3% degradation in tail latency when hierarchy was removed. We formulated the problem as a constrained Markov Decision Process with a multi-objective reward function that balances technical performance with business outcomes, enabling simultaneous optimization of latency, throughput, and business value while respecting operational constraints. Our comprehensive evaluation demonstrated that AdaptiveGate reduces tail latency by 42.3%, improves throughput by 35.7%, and enhances system resilience compared to state-of-the-art approaches. These technical improvements translate to substantial business benefits, including a 4.3% increase in platform revenue and 18% reduction in infrastructure costs, establishing that reinforcement learning can be effectively deployed in mission-critical environments where performance directly impacts business outcomes.

Despite these achievements, several significant challenges remain that warrant dedicated future research exploration, requiring specific technical approaches rather than general methodological suggestions. Training stability in non-stationary environments represents a fundamental challenge where we propose concrete solutions including Model-Agnostic Meta-Learning (MAML) variants specifically designed for traffic pattern recognition tasks. These MAML adaptations would incorporate temporal context encoders that capture advertising-specific cyclical patterns, enabling rapid adaptation to new traffic distributions within 10-20 gradient steps. Continual learning approaches using Elastic Weight Consolidation (EWC) can prevent catastrophic forgetting of previously learned traffic management strategies by identifying and preserving critical neural network parameters associated with stable routing policies. Online adaptation mechanisms combining uncertainty estimation with constrained exploration would enable real-time policy adjustment while maintaining safety bounds through confidence-weighted action selection and rollback capabilities when performance degrades beyond predetermined thresholds. Cold-start performance challenges require specialized transfer learning methodologies beyond generic domain adaptation approaches. Cross-domain policy distillation can leverage knowledge from similar services by training student networks to mimic teacher policies from established services while fine-tuning for new service characteristics through supervised learning on routing decisions. Uncertainty-quantified exploration strategies using Thompson sampling with service-specific priors would enable principled exploration in data-scarce scenarios, where prior distributions are constructed from historical performance of similar services and updated through Bayesian inference as new observations become available. Progressive neural architecture adaptation would begin with simplified models containing 50-100 parameters for initial deployment, automatically expanding to full complexity as sufficient training data accumulates, with architecture selection guided by information-theoretic criteria that balance model capacity with available data.

REFERENCES

[1] S. Yuan, J. Wang, and X. Zhao, "Real-time bidding for online advertising: Measurement and analysis," in *Proc. 7th Int. Workshop Data Mining Online Advertising*, Aug. 2013, pp. 1–8.

[2] J. Wang, W. Zhang, and S. Yuan, "Display advertising with real-time bidding (RTB) and behavioural targeting," *Found. Trends Inf. Retr.*, vol. 11, no. 4, pp. 297–435, 2017.

[3] K. Kenthapadi, B. Le, and G. Venkataraman, "Personalized job recommendation system at LinkedIn: Practical challenges and lessons learned," in *Proc. 11th ACM Conf. Recommender Syst.*, Aug. 2017, pp. 346–347.

[4] W. Zhang, S. Yuan, J. Wang, and X. Shen, "Real-time bidding benchmarking with iPinYou dataset," 2014, *arXiv:1407.7073*.

[5] H.-T. Cheng, L. Koc, J. Harmsen, T. Shaked, T. Chandra, H. Aradhye, G. Anderson, G. Corrado, W. Chai, M. Ispir, R. Anil, Z. Haque, L. Hong, V. Jain, X. Liu, and H. Shah, "Wide & deep learning for recommender systems," in *Proc. 1st Workshop Deep Learn. Recommender Syst.*, Sep. 2016, pp. 7–10.

[6] G. Zhou, X. Zhu, C. Song, Y. Fan, Z. Han, X. Ma, Y. Yan, J. Jin, H. Li, and K. Gai, "Deep interest network for click-through rate prediction," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2018, pp. 1059–1068.

[7] X. He, J. Pan, O. Jin, T. Xu, B. Liu, T. Xu, Y. Shi, A. Atallah, R. Herbrich, S. Bowers, and J. Q. Candela, "Practical lessons from predicting clicks on ads at Facebook," in *Proc. 8th Int. Workshop Data Mining Online Advertising*, Aug. 2014, pp. 1–9.

[8] O. Chapelle, E. Manavoglu, and R. Rosales, "Simple and scalable response prediction for display advertising," *ACM Trans. Intell. Syst. Technol.*, vol. 5, no. 4, pp. 1–34, Jan. 2015.

[9] L. Liu, J. Liu, and D. Tao, "Variance reduced methods for non-convex composition optimization," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 9, pp. 5813–5825, Sep. 2022.

[10] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present Ulterior Softw. Eng.*, 2017, pp. 195–216.

[11] D. Saxena and B. Bhowmik, "Ways of balancing load in microservice architecture," in *Proc. Int. Conf. VLSI*, 2024, pp. 379–396.

[12] S. Rahman, T. Ahmed, M. Huynh, M. Tornatore, and B. Mukherjee, "Auto-scaling network service chains using machine learning and negotiation game," *IEEE Trans. Netw. Service Manage.*, vol. 17, no. 3, pp. 1322–1336, Sep. 2020.

[13] A. Gandhi, M. Harchol-Balter, R. Raghunathan, and M. A. Kozuch, "AutoScale: Dynamic, robust capacity management for multi-tier data centers," *ACM Trans. Comput. Syst.*, vol. 30, no. 4, pp. 1–26, Nov. 2012.

[14] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, p. 1, 2015.

[15] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.

[16] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 26–38, Nov. 2017.

[17] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, "Mastering the game of go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, Oct. 2017.

[18] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proc. 15th ACM workshop hot topics Netw.*, 2016, pp. 50–56.

[19] G. Tesauro, N. K. Jong, R. Das, and M. N. Bennani, "A hybrid reinforcement learning approach to autonomic resource allocation," in *Proc. IEEE Int. Conf. Autonomic Comput.*, Jun. 2006, pp. 65–73.

[20] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.

[21] G. Dulac-Arnold, D. Mankowitz, and T. Hester, "Challenges of real-world reinforcement learning," 2019, *arXiv:1904.12901*.

[22] N. Lazic, C. Boutilier, T. Lu, E. Wong, B. Roy, M. Ryu, and G. Imwalle, "Data center cooling using model-predictive control," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 3814–3823.

[23] Z. Xu, J. Tang, J. Meng, W. Zhang, Y. Wang, C. H. Liu, and D. Yang, "Experience-driven networking: A deep reinforcement learning based approach," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2018, pp. 1871–1879.

[24] S. Fujimoto, H. v. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1587–1596.

[25] C. Richardson, *Microservices Patterns: With Examples in Java*. New York, NY, USA: Simon and Schuster, 2018.

[26] P. Jamshidi, C. Pahl, N. C. Mendonça, J. Lewis, and S. Tilkov, "Microservices: The journey so far and challenges ahead," *IEEE Softw.*, vol. 35, no. 3, pp. 24–35, May 2018.

[27] J. Wan, L. Liu, and J. Guo, "Dynamic request routing for online video-on-demand service: A Markov decision process approach," *Math. Problems Eng.*, vol. 2014, no. 1, Jan. 2014, Art. no. 920829.

[28] M. Nardelli, S. Nastic, S. Dustdar, M. Villari, and R. Ranjan, "Osmotic flow: Osmotic computing + IoT workflow," *IEEE Cloud Comput.*, vol. 4, no. 2, pp. 68–75, Mar. 2017.

[29] C. Guerrero, I. Lera, and C. Juiz, "Genetic algorithm for multi-objective optimization of container allocation in cloud architecture," *J. Grid Comput.*, vol. 16, no. 1, pp. 113–135, Mar. 2018.

[30] Q. Yang, W.-N. Chen, T. Gu, H. Zhang, H. Yuan, S. Kwong, and J. Zhang, "A distributed swarm optimizer with adaptive communication for large-scale optimization," *IEEE Trans. Cybern.*, vol. 50, no. 7, pp. 3393–3408, Jul. 2020.

[31] S. D. Marcus, A. Shapiro, and C. Giladi, "Enhancing quadruped robot walking on unstructured terrains: A combination of stable blind gait and deep reinforcement learning," *Electronics*, vol. 14, no. 7, p. 1431, Apr. 2025.

[32] A. Alelyani, A. Datta, and G. M. Hassan, "Optimizing cloud performance: A microservice scheduling strategy for enhanced fault-tolerance, reduced network traffic, and lower latency," *IEEE Access*, vol. 12, pp. 35135–35153, 2024.

[33] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in *Proc. ACM Special Interest Group Data Commun.*, Aug. 2019, pp. 270–288.

[34] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proc. 16th ACM workshop hot topics Netw.*, 2017, pp. 185–191.

[35] F. Rossi, M. Nardelli, and V. Cardellini, "Horizontal and vertical scaling of container-based applications using reinforcement learning," in *Proc. IEEE 12th Int. Conf. Cloud Comput. (CLOUD)*, Jul. 2019, pp. 329–338.

[36] S. Joshi, N. K. Pandey, and A. K. Mishra, "Reinforcement learning based auto scaling strategy used in cloud environment: State of art," in *Proc. IEEE 13th Int. Conf. Commun. Syst. Netw. Technol. (CSNT)*, Apr. 2024, pp. 731–736.

[37] S. Pateria, B. Subagdja, A.-H. Tan, and C. Quek, "Hierarchical reinforcement learning: A comprehensive survey," *ACM Comput. Surveys*, vol. 54, no. 5, pp. 1–35, Jun. 2022.

[38] P. Bacon, J. Harb, and D. Precup, "The option-critic architecture," in *Proc. AAAI Conf. Artif. Intell.*, 2017, vol. 31, no. 1, pp. 1726–1734.

[39] O. Nachum, S. Gu, H. Lee, and S. Levine, "Data-efficient hierarchical reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018.

[40] K. Zhang, Z. Yang, and T. Başar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," in *Handbook of Reinforcement Learning and Control*. Switzerland: Springer, 2021, pp. 321–384.

[41] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017.

[42] T. Rashid, M. Samvelyan, C. S. D. Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multi-agent reinforcement learning," *J. Mach. Learn. Res.*, vol. 21, no. 178, pp. 1–51, 2020.

[43] J. Wang, S. Yuan, and W. Zhang, "Real-time bidding based display advertising: Mechanisms and algorithms," in *Proc. Eur. Conf. Inf. Retr.*, Padua, Italy, Mar. 2016, pp. 897–901.

[44] Y. Chen, P. Berkhin, B. Anderson, and N. R. Devanur, "Real-time bidding algorithms for performance-based display ad allocation," in *Proc. 17th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2011, pp. 1307–1315.

[45] K. Ren, J. Qin, L. Zheng, Z. Yang, W. Zhang, and Y. Yu, "Deep landscape forecasting for real-time bidding advertising," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 363–372.

[46] X. Yang, D. Sun, R. Zhu, T. Deng, Z. Guo, Z. Ding, S. Qin, and Y. Zhu, "AiAds: Automated and intelligent advertising system for sponsored search," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 1881–1890.

[47] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, "Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms," in *Proc. 26th Symp. Operating Syst. Princ.*, Oct. 2017, pp. 153–167.

[48] K. Rzadca, P. Findeisen, J. Swiderski, P. Zych, P. Broniek, J. Kusmierek, P. Nowak, B. Strack, P. Witusowski, S. Hand, and J. Wilkes, "Autopilot: Workload autoscaling at Google," in *Proc. 15th Eur. Conf. Comput. Syst.*, Apr. 2020, pp. 1–16.

[49] C.-Y. Hong et al., "B4 and after: Managing hierarchy, partitioning, and asymmetry for availability and scale in Google's software-defined WAN," in *Proc. Conf. ACM Special Interest Group Data Commun.*, Aug. 2018, pp. 74–87.

[50] R. Veisllari, S. Bjornstad, J. P. Braute, K. Bozorgebrahimi, and C. Raffaelli, "Field-trial demonstration of cost efficient sub-wavelength service through integrated packet/circuit hybrid network [invited]," *J. Opt. Commun. Netw.*, vol. 7, no. 3, pp. A379–A387, Mar. 2015.

[51] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul, "Neo: A learned query optimizer," 2019, *arXiv:1904.03711*.

[52] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.

[53] Y. Chow, M. Ghavamzadeh, L. Janson, and M. Pavone, "Risk-constrained reinforcement learning with percentile risk criteria," *J. Mach. Learn. Res.*, vol. 18, no. 167, pp. 1–51, 2017.

[54] M. Plappert, R. Houthooft, P. Dhariwal, S. Sidor, R. Y. Chen, X. Chen, T. Asfour, P. Abbeel, and M. Andrychowicz, "Parameter space noise for exploration," 2017, *arXiv:1706.01905*.

[55] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," 2020, *arXiv:2005.01643*.

[56] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," 2015, *arXiv:1511.05952*.

[57] C. Guerrero, I. Lera, and C. Juiz, "Resource optimization of container orchestration: A case study in multi-cloud microservices-based applications," *J. Supercomput.*, vol. 74, no. 7, pp. 2956–2983, Jul. 2018.

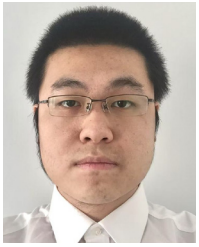
[58] Y. Niu, F. Liu, and Z. Li, "Load balancing across microservices," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2018, pp. 198–206.

[59] W. Zhang, Y. Rong, J. Wang, T. Zhu, and X. Wang, "Feedback control of real-time display advertising," in *Proc. 9th ACM Int. Conf. Web Search Data Mining*, Feb. 2016, pp. 407–416.
[60] R. Paes Leme, M. Pal, and S. Vassilvitskii, "A field guide to personalized reserve prices," in *Proc. 25th Int. Conf. World Wide Web*, Apr. 2016, pp. 1093–1102.

ENKAI JI received the bachelor's and Master of Science degrees in computer science from Rutgers University. He has a robust background in software engineering, web development, and data processing, with prior experience as a Research Assistant and a Full-Stack Developer. His research interests include scalable system architecture and artificial intelligence, including deep learning applications in advertising technologies and e-commerce platforms.

YIHAN WANG received the M.S.E. degree in computer and information science from the University of Pennsylvania. She currently focuses on CPU resource management for database systems and contributes to the development of OS-level resource control mechanisms. Her research interests include operating system design, performance optimization, systems engineering, and the application of AI techniques in intelligent resource scheduling and cloud infrastructure. She is also passionate about exploring AI's integration across industry use cases to enhance computing efficiency and scalability.

SUCHUAN XING received the M.S. degree in electrical and computer engineering from Duke University, in 2023. He is also deeply interested in the application of artificial intelligence across various industries, including the IoT, smart systems, and software infrastructure. His research interests include distributed tracing, reinforcement learning for API traffic management, cross-platform software architectures, and system performance optimization.



JIANIAN JIN received the Master of Science degree in data science from Columbia University. He is currently a Software Development Engineer. His professional expertise includes building scalable big data applications, automating testing solutions, and deploying serverless computing technologies. His research interests include cloud computing, big data analytics, deep leaning, and software engineering, with a strong emphasis on leveraging advanced technologies to enhance efficiency and cost-effectiveness.